# Unit 3

# Data Link Layer

# Syllabus

**Unit-3: Data Link Layer**

Error Detection and Correction: Types of Errors, Detection, Error Correction.

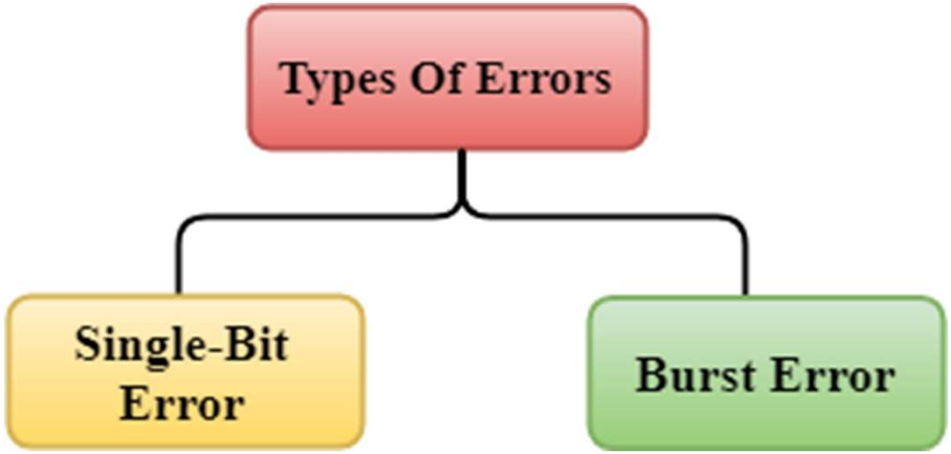
Data Link Control and Protocols: Flow and Error Control, Stop-and-wait ARQ, Go-Back-N ARQ, Selective Repeat ARQ, HDLC, PPP.

Multiple Accesses: Random Access, Controlled Access, Channelization. Local area Network: Standard Ethernet, Wireless LANs: IEEE 802.11.

# Error Detection and Correction

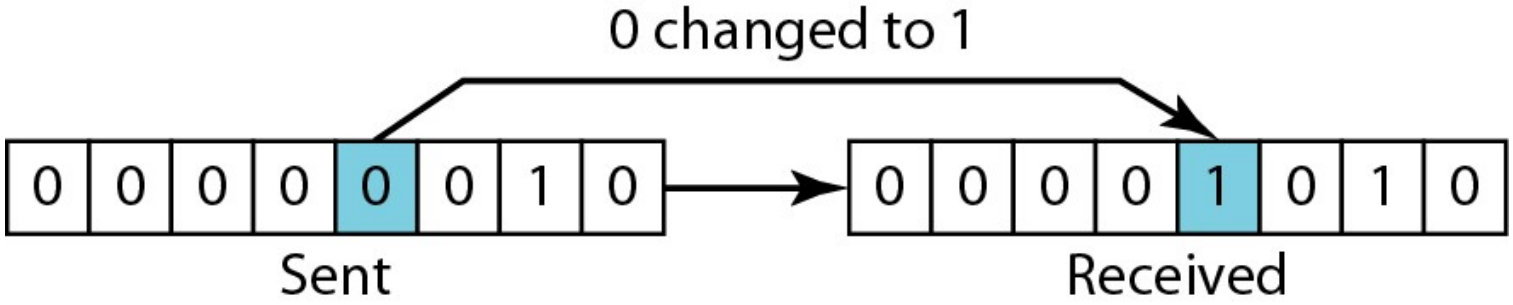
# Error Detection

- When data is transmitted from one device to another device, data can be corrupted during transmission.
- An error is a situation when the message received at the receiver end is not identical to the message transmitted.



# Single-Bit Error

The term *single-bit error* means that only 1 bit of a given data unit is changed from 1 to 0 or from 0 to 1.



# Burst Error

- A burst error means that 2 or more bits in the data unit have changed.

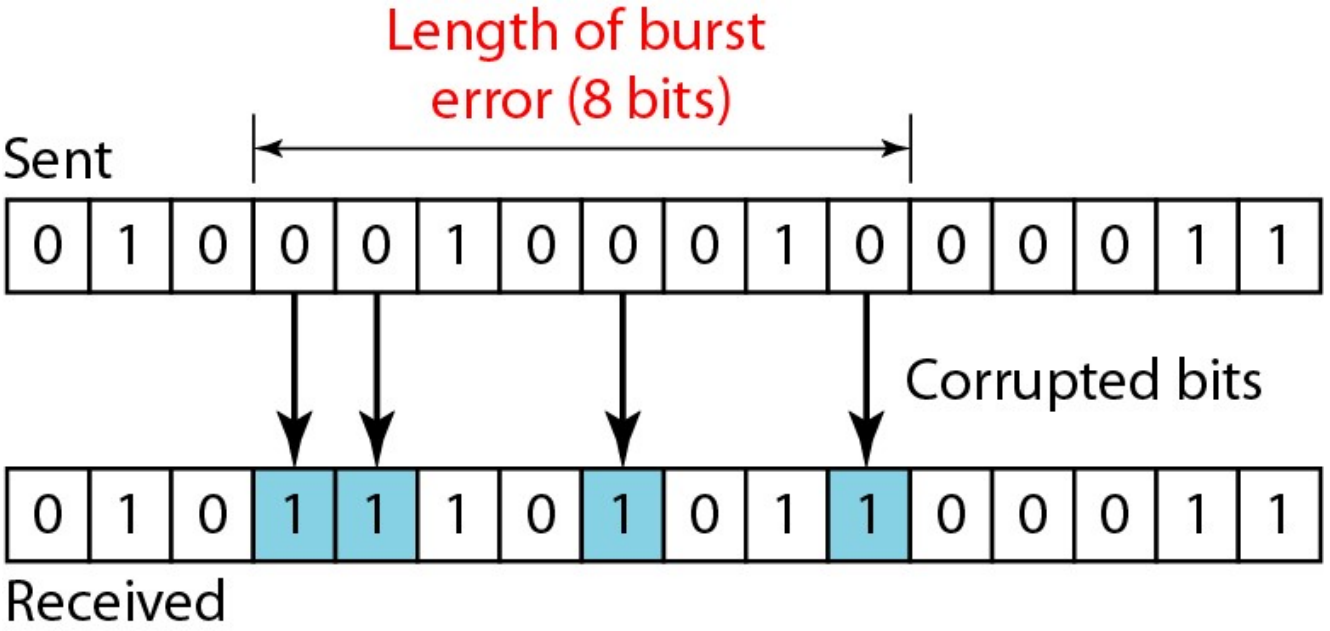



*Fig : Burst error of length 8*

# Block Coding

*In block coding, we divide our message into blocks, each of k bits, called datawords. We add r redundant bits to each block to make the length n = k + r. The resulting n-bit blocks are called codewords.*

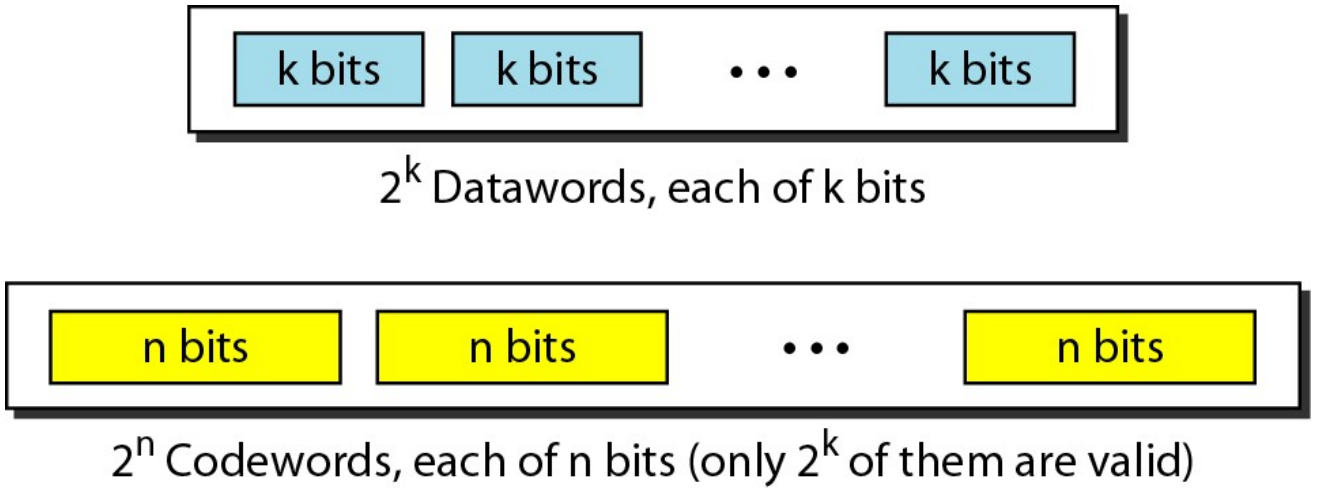
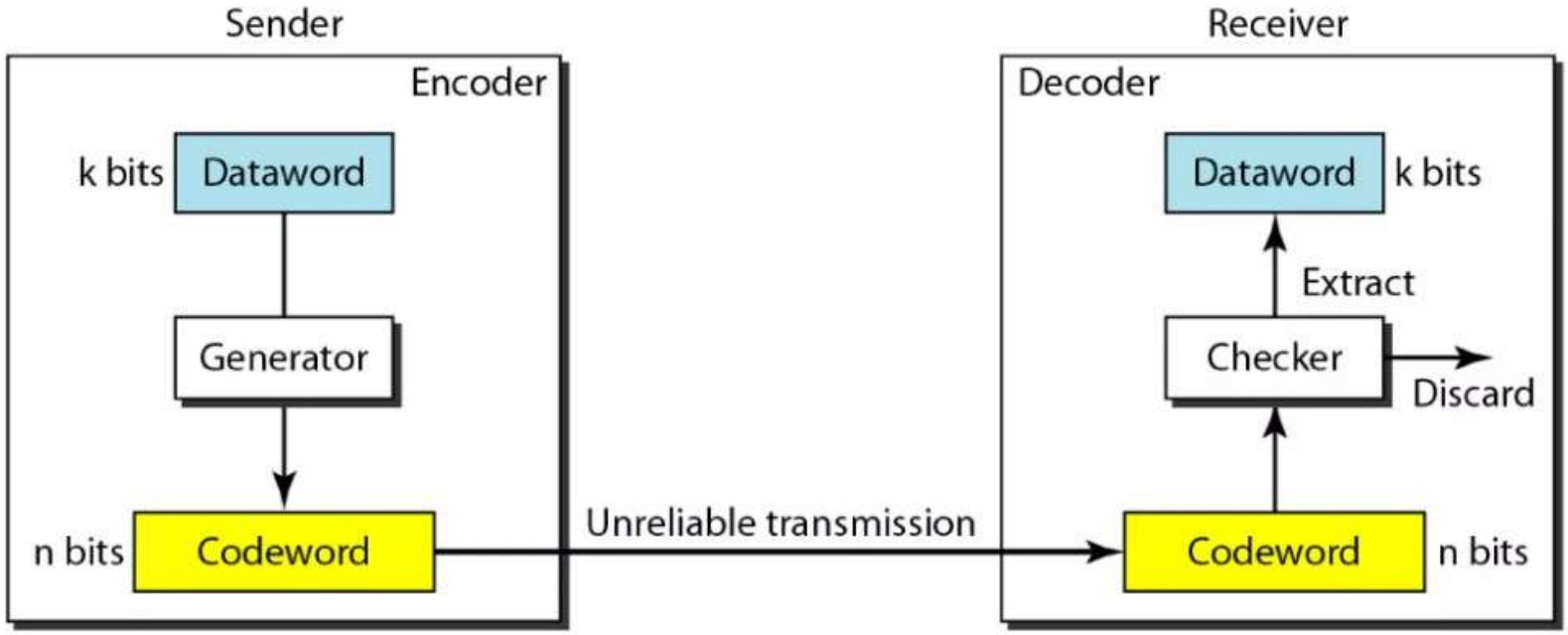



*Fig: Datawords and codewords in block coding*

# *Process of error detection in block coding*



# Block Coding

Let us assume that k=2 and n=3. Table 1 shows the list of data words and codewords.

*Table 1: A code for error detection*

| *Datawords* | *Codewords* |
|---|---|
| 00 | 000 |
| 01 | 011 |
| 10 | 101 |
| 11 | 110 |

# Valid & Invalid Code Word

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases. Consider the following cases.

1. The receiver receives 011. It is a valid codeword. The receiver extracts the data word 01 from it.
2. The codeword is corrupted during transmission and 111 is received. This is not a valid codeword and is discarded.
3. The codeword is corrupted during transmission and 000 is received (the right two bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the datawords 00. Two corrupted bits have made the error undetectable.

# Hamming Distance

The Hamming distance between two words is the number of differences between corresponding bits.

*Hamming distance between two pairs of words.*

1. *The Hamming distance d(000,011) is 2*

$000 \oplus 011$ is 011 (two 1s)

2. *The Hamming distance d(10101, 11110) is 3*

$10101 \oplus 11110$ is 01011 (three 1s)

| Two Input XOR gate | | |
|---|---|---|
| A | B | $Y = A \oplus B$ |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

# Minimum Hamming Distance

**The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.**

***Find the minimum Hamming distance of the coding***

| *Datawords* | *Codewords* |
|---|---|
| 00 | 000 |
| 01 | 011 |
| 10 | 101 |
| 11 | 110 |

*Solution*

*We first find all Hamming distances.*

| | | | |
|---|---|---|---|
| $d(000, 011) = 2$ | $d(000, 101) = 2$ | $d(000, 110) = 2$ | $d(011, 101) = 2$ |
| $d(011, 110) = 2$ | $d(101, 110) = 2$ | | |

*The $d_{min}$ in this case is 2.*

***Find the minimum Hamming distance of the coding scheme***

| *Dataword* | *Codeword* |
|---|---|
| 00 | 00000 |
| 01 | 01011 |
| 10 | 10101 |
| 11 | 11110 |

*Find the minimum Hamming distance of the coding scheme*

| *Dataword* | *Codeword* |
|---|---|
| 00 | 00000 |
| 01 | 01011 |
| 10 | 10101 |
| 11 | 11110 |

## *Solution*

*We first find all the Hamming distances.*

| | | |
|---|---|---|
| $d(00000, 01011) = 3$ | $d(00000, 10101) = 3$ | $d(00000, 11110) = 4$ |
| $d(01011, 10101) = 4$ | $d(01011, 11110) = 3$ | $d(10101, 11110) = 3$ |

*The* $d_{min}$ *in this case is 3.*

# LINEAR BLOCK CODES

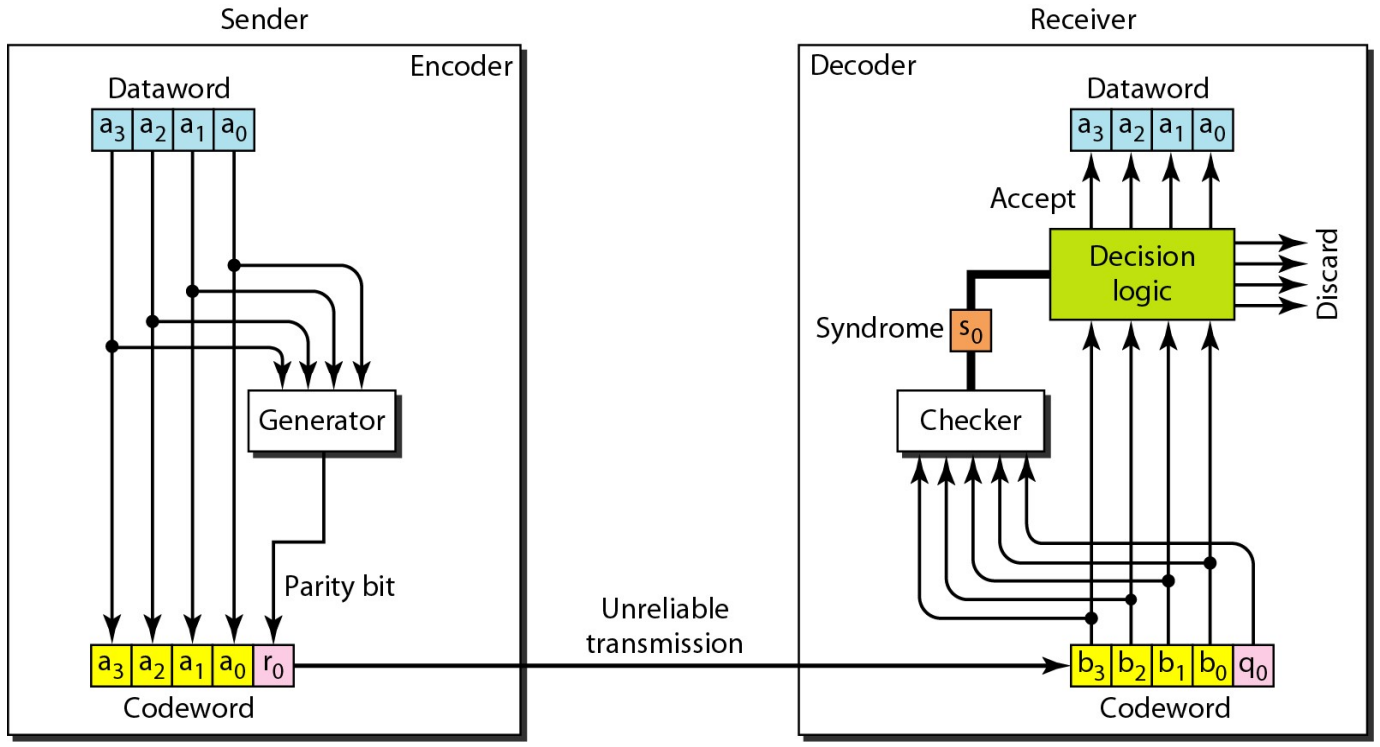
•A **Linear Block Code** is a type of **block code** in which every valid codeword is a **linear combination** of the message bits.
•It maps **k information bits** into an **n-bit codeword** by adding **(n−k) parity bits**.
•The modulo-2 sum (XOR) of any two valid codewords is also a valid codeword.

Examples of Linear Block Codes
•Single Parity Check Code
•Hamming Code
•Cyclic Code
•Reed–Solomon Code
•BCH Code
•Golay Code

# Single Parity Check Code

- In this code, a *r*-bit dataword is changed to an n-bit codeword where $n = k + 1$.
- The extra bit is called the *parity bit*, is selected to make the total number of 1's in the codeword even (for *even parity*) or odd (for *odd parity*) .

- If the number of 1's is even, the result is 0;
- if the number of 1s is odd, the result is 1.
- In both cases, the total number of 1s in the codeword is even.



# Hamming Code

•**Hamming code** is a linear block code which adds parity bits to the original data bits to **detect and correct** transmission **errors**.

•Hamming code can **correct single-bit errors** and **detect double-bit errors** (with an additional overall parity bit).

### Number of Parity Bits

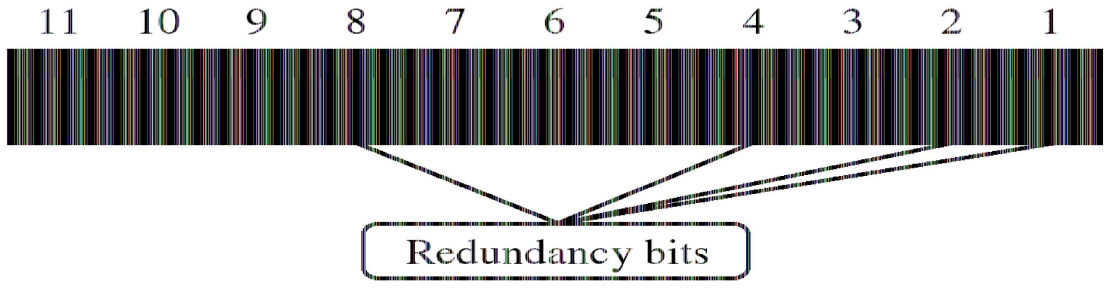
$$2^r \geq k + r + 1$$

| Parameter | Description | Example: Hamming (7,4) |
|---|---|---|
| k | Data bits | 4 |
| r | Parity/redundant bits | 3 |
| n | Total codeword bits | 7 |
| (n,k) | Code notation | **(7,4)** |

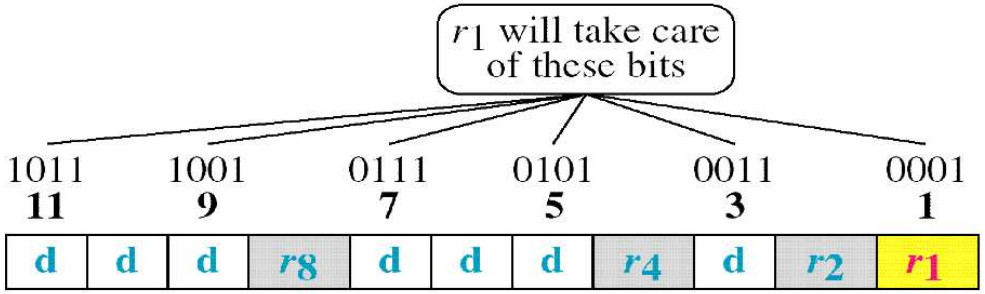
### Placement of Data & Parity Bits

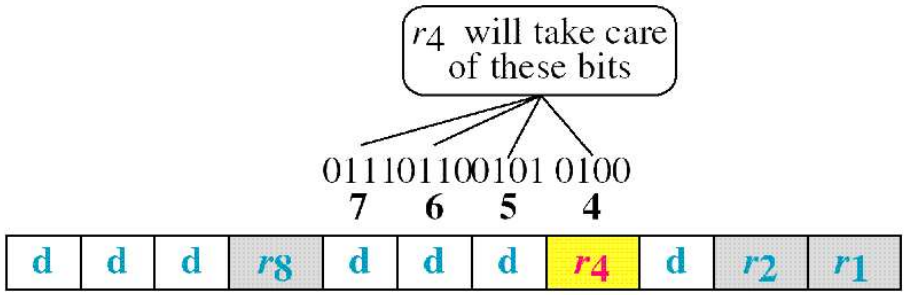
•Place parity bits at positions: **1, 2, 4, 8, ...** (powers of 2)

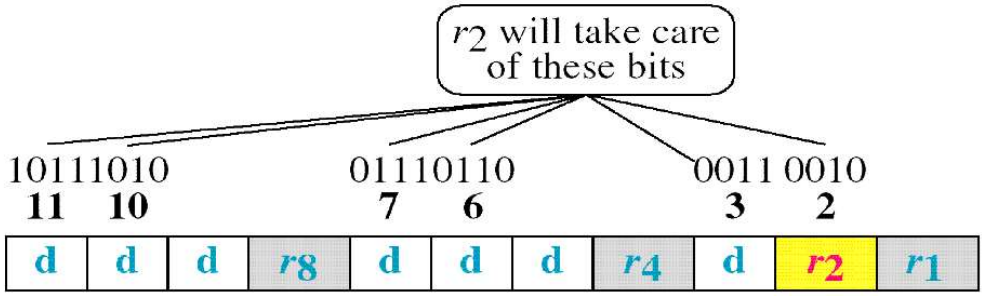
•Insert data bits in the remaining positions.

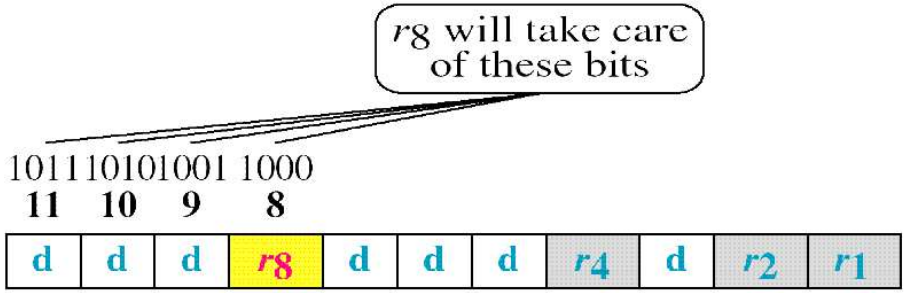


# Hamming Code




- **P1:** checks **1 bit**, then **skips 1 bit**, and repeats this pattern until the end of the codeword.
- **P2:** checks **2 consecutive bits**, then **skips 2 bits**, and repeats this pattern throughout the codeword.
- **P4:** checks **4 consecutive bits**, then **skips 4 bits**, and continues this alternating pattern until the last bit.
- **P8:** checks **8 consecutive bits**, then **skips 8 bits**, and repeats this process across the entire codeword.

**A 4-bit data block D=[1 1 0 1] is to be transmitted using the Hamming (7,4) code with even parity.**

**(i)** Construct the corresponding 7-bit Hamming codeword by determining the parity bits P1, P2, and P4.

**(ii)** Assume that during transmission, the **4th bit** of the codeword is corrupted. Determine the received codeword and use **syndrome decoding** to detect and locate the error.

**(iii)** Correct the erroneous bit and recover the original 4-bit data block.

## (i) Construct the corresponding 7-bit Hamming codeword by determining the parity bits P1, P2, and P4

| Bit Position | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|
| Bit Type | $d_7$ | $d_6$ | $d_5$ | $P_4$ | $d_3$ | $P_2$ | $P_1$ |
| Data Word (without parity) | 1 | 1 | 0 | | 1 | | |
| $P_1 = d_3 \oplus d_5 \oplus d_7$ | 1 | | 0 | | 1 | | 0 |
| $P_2 = d_3 \oplus d_6 \oplus d_7$ | 1 | 1 | | | 1 | 1 | |
| $P_4 = d_5 \oplus d_6 \oplus d_7$ | 1 | 1 | 0 | 0 | | | |
| Final Codeword | 1 | 1 | 0 | 0 | 1 | 1 | 0 |

$$P_1 = d_3 \oplus d_5 \oplus d_7 = 1 \oplus 0 \oplus 1 = 0$$
$$P_2 = d_3 \oplus d_6 \oplus d_7 = 1 \oplus 1 \oplus 1 = 1$$
$$P_4 = d_5 \oplus d_6 \oplus d_7 = 0 \oplus 1 \oplus 1 = 0$$

**Final Code Word: 1100110**

**(ii)** Assume that during transmission, the **4th bit** of the codeword is corrupted. Determine the received codeword and use **syndrome decoding** to detect and locate the error.

Original: 1100110
Bit 4 changes from **0 → 1**
Codeword: 1101110

| **Bit Position** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **Parity Check (XOR)** | **Syndrome** |
|---|---|---|---|---|---|---|---|---|---|
| **Bit Type** | $d_7$ | $d_6$ | $d_5$ | $P_4$ | $d_3$ | $P_2$ | $P_1$ | | |
| **Received Codeword** | **1** | **1** | **0** | **1** | **1** | **1** | **0** | | |
| $S_1$ (Checks 1,3,5,7) | **1** | | **0** | | **1** | | **0** | $0 \oplus 1 \oplus 0 \oplus 1 = \mathbf{0}$ | $S_1 = 0$ |
| $S_2$ (Checks 2,3,6,7) | **1** | **1** | | | **1** | **1** | | $1 \oplus 1 \oplus 1 \oplus 1 = \mathbf{0}$ | $S_2 = 0$ |
| $S_4$ (Checks 4,5,6,7) | **1** | **1** | **0** | **1** | | | | $1 \oplus 0 \oplus 1 \oplus 1 = \mathbf{1}$ | $S_4 = 1$ |

$$S_4S_2S_1 = 100_2 = 4_{10}$$ **Hence, the error is at bit position 4.**

**(iii) Correct the erroneous bit and recover the original 4-bit data block.**

| Bit Position | **7** | **6** | **5** | **4** | **3** | **2** | **1** |
|---|---|---|---|---|---|---|---|
| Received | 1 | 1 | 0 | 1 | 1 | 1 | 0 |
| Corrected | 1 | 1 | 0 | 0 | 1 | 1 | 0 |

# Cyclic Codes

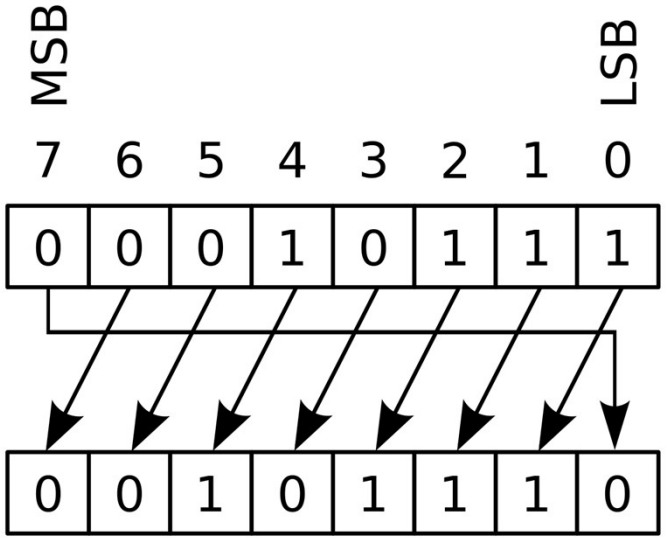
*Cyclic codes* are special linear block codes with one extra property. In a *cyclic code*, if a codeword is cyclically shifted (rotated), the result is another codeword.

*Codeword1* = 1011000

*Codeword2* = 0110001

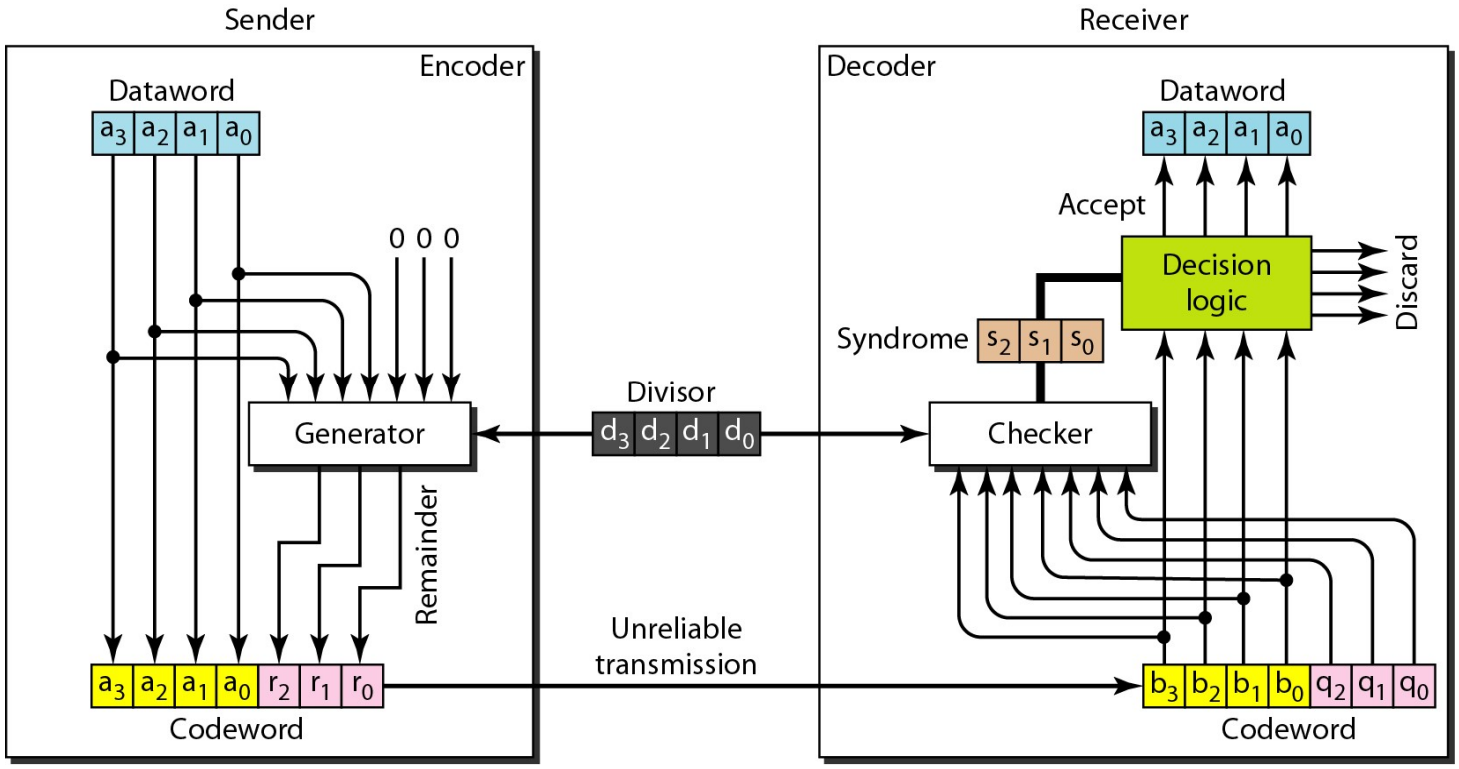
$b_0 = a_6 \quad b_6 = a_5 \quad b_5 = a_4 \quad b_4 = a_3$

$b_3 = a_2 \quad b_2 = a_1 \quad b_1 = a_0$



# CRC Encoder and Decoder

- In the encoder, the dataword has *k* bits (4 here); the codeword has *n* bits (7 here).
- The size of the dataword is augmented by adding *n* - *k* 0s to the right-hand side of the word.
- The *n*-bit result is fed into the generator.
- The divisor in a cyclic code is normally called the generator polynomial.




The generator uses a divisor of size = ***n - k*** + **1** (4 here), predefined and agreed upon.

Figure *Division in CRC encoder*

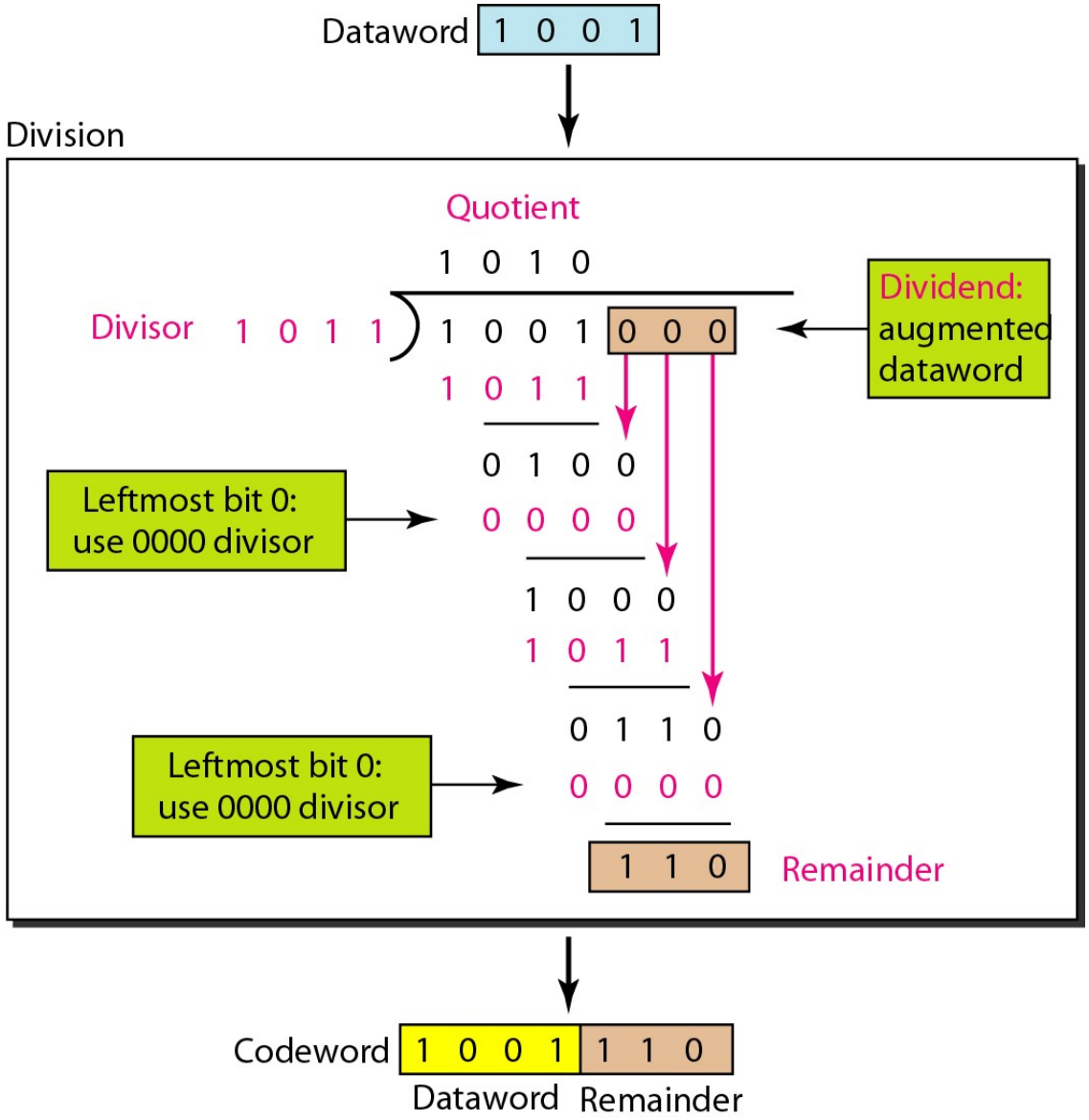
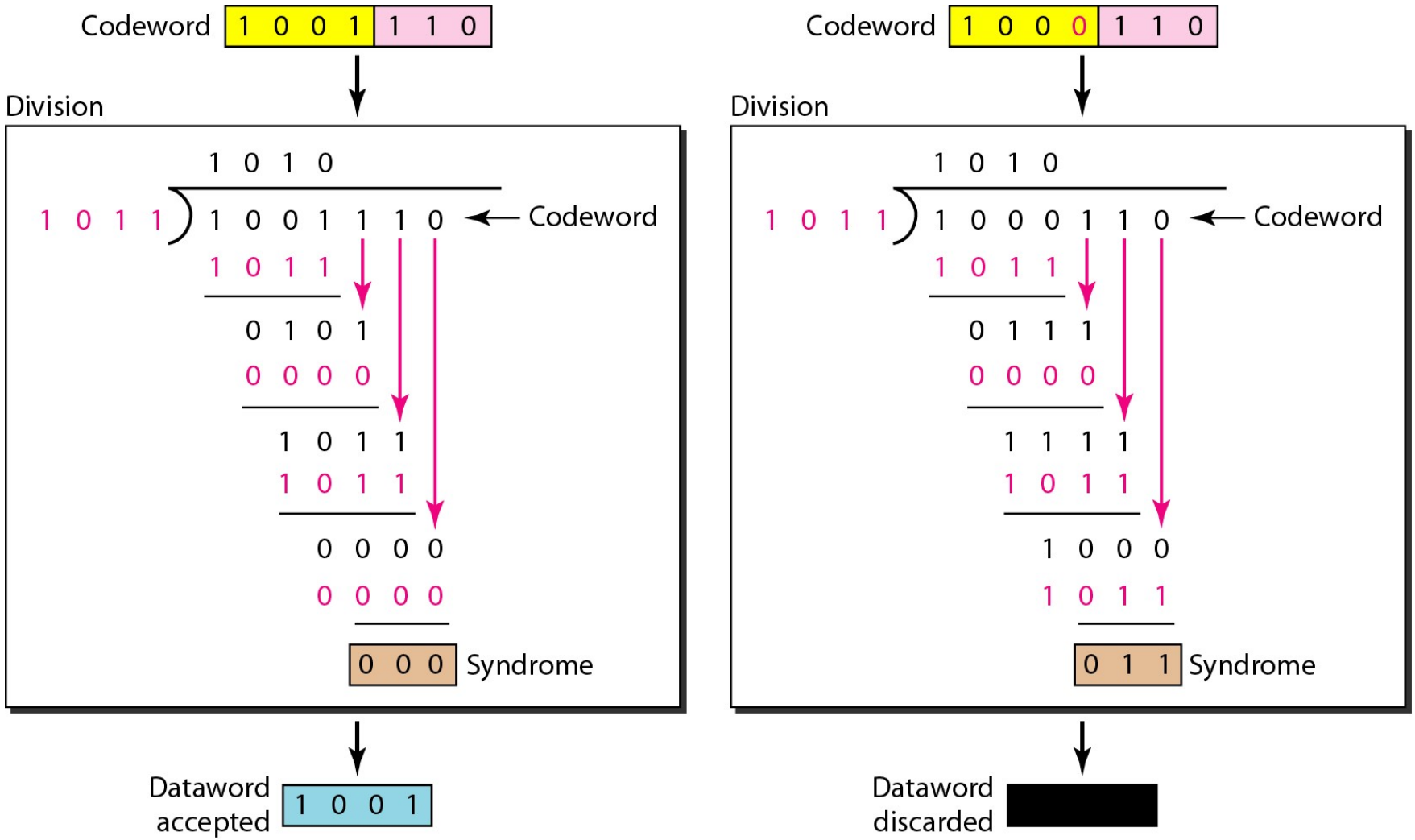
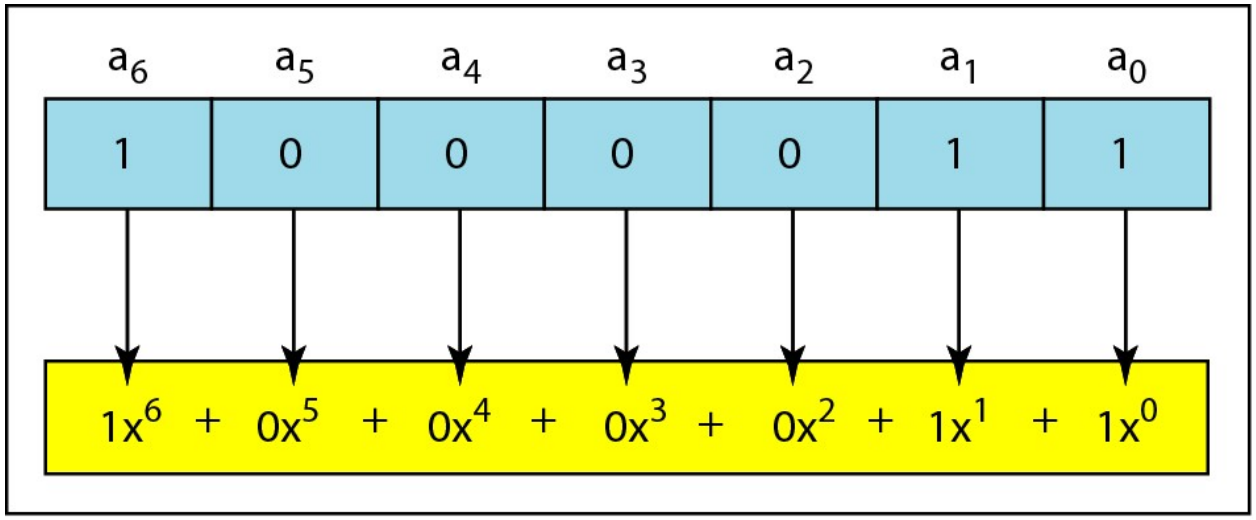


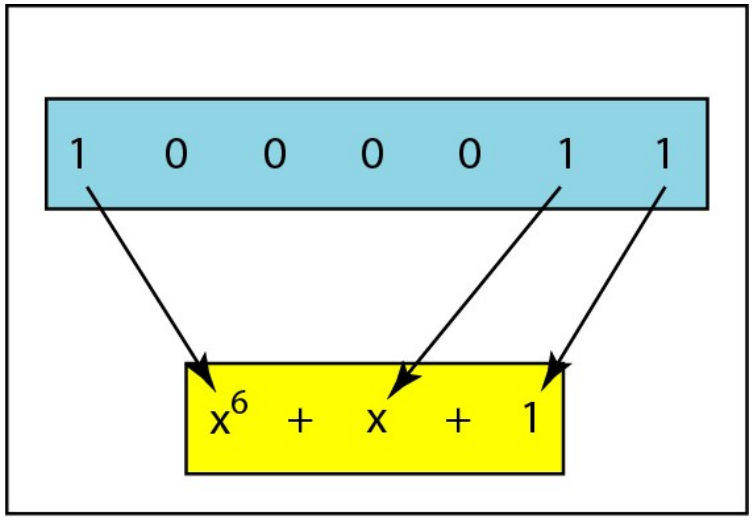
Figure *Division in the CRC decoder for two cases*



# *A polynomial to represent a binary word*

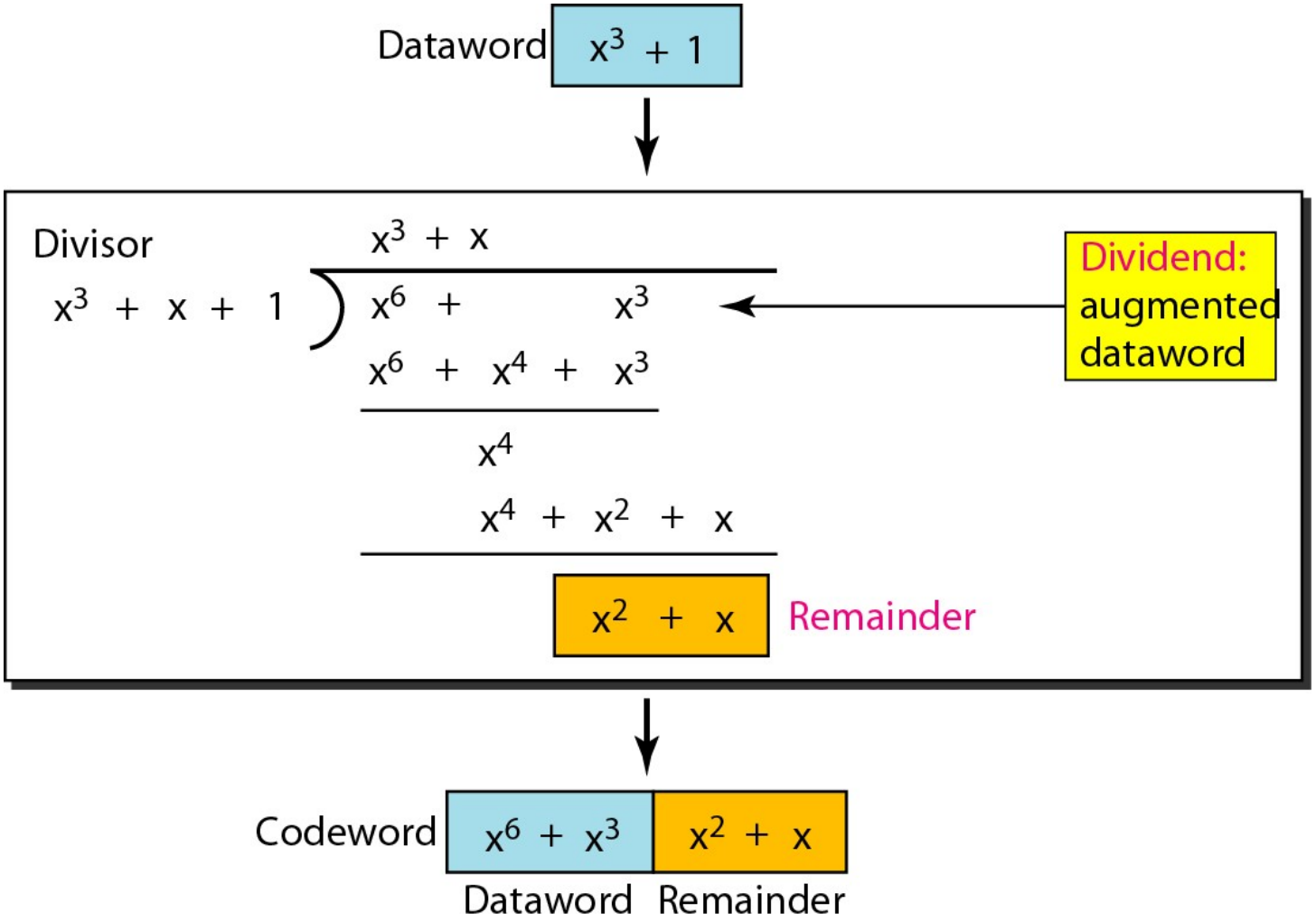



a. Binary pattern and polynomial




b. Short form

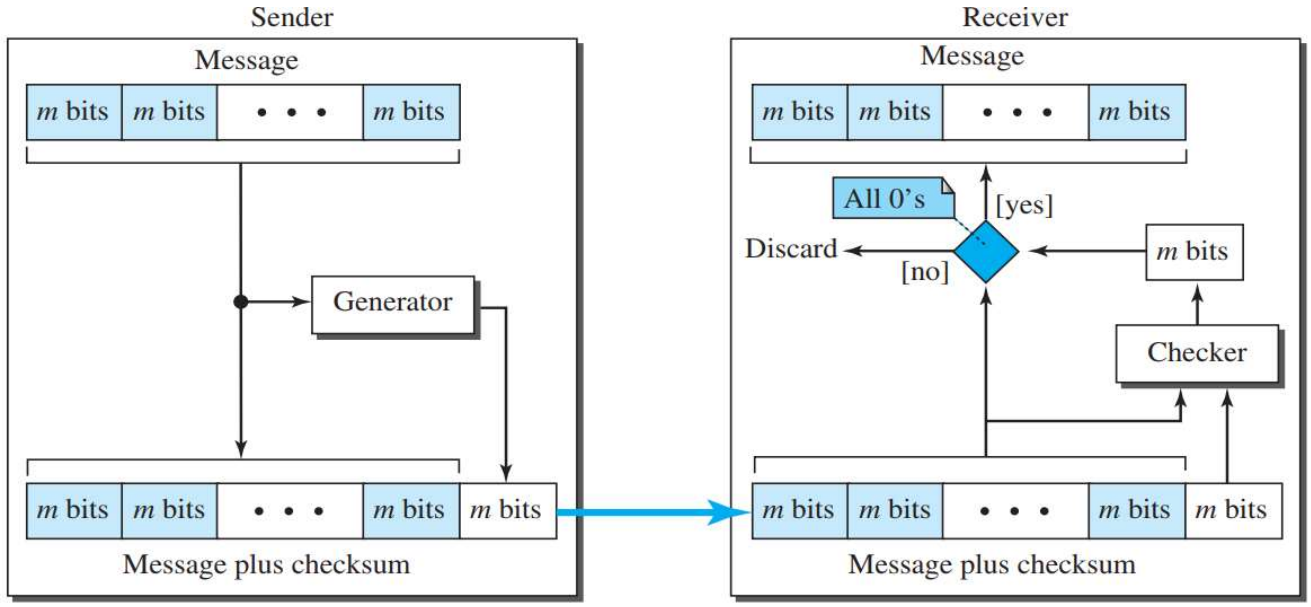
Figure *CRC division using polynomials*



# *Checksum*

**Checksum** is an error-detecting technique that can be applied to a message of any length. In the Internet, the checksum technique is mostly used at the network and transport layer rather than the data-link layer.

At the source, the message is first divided into *m*-bit units. The generator then creates an extra *m*-bit unit called the ***checksum,*** which is sent with the message.



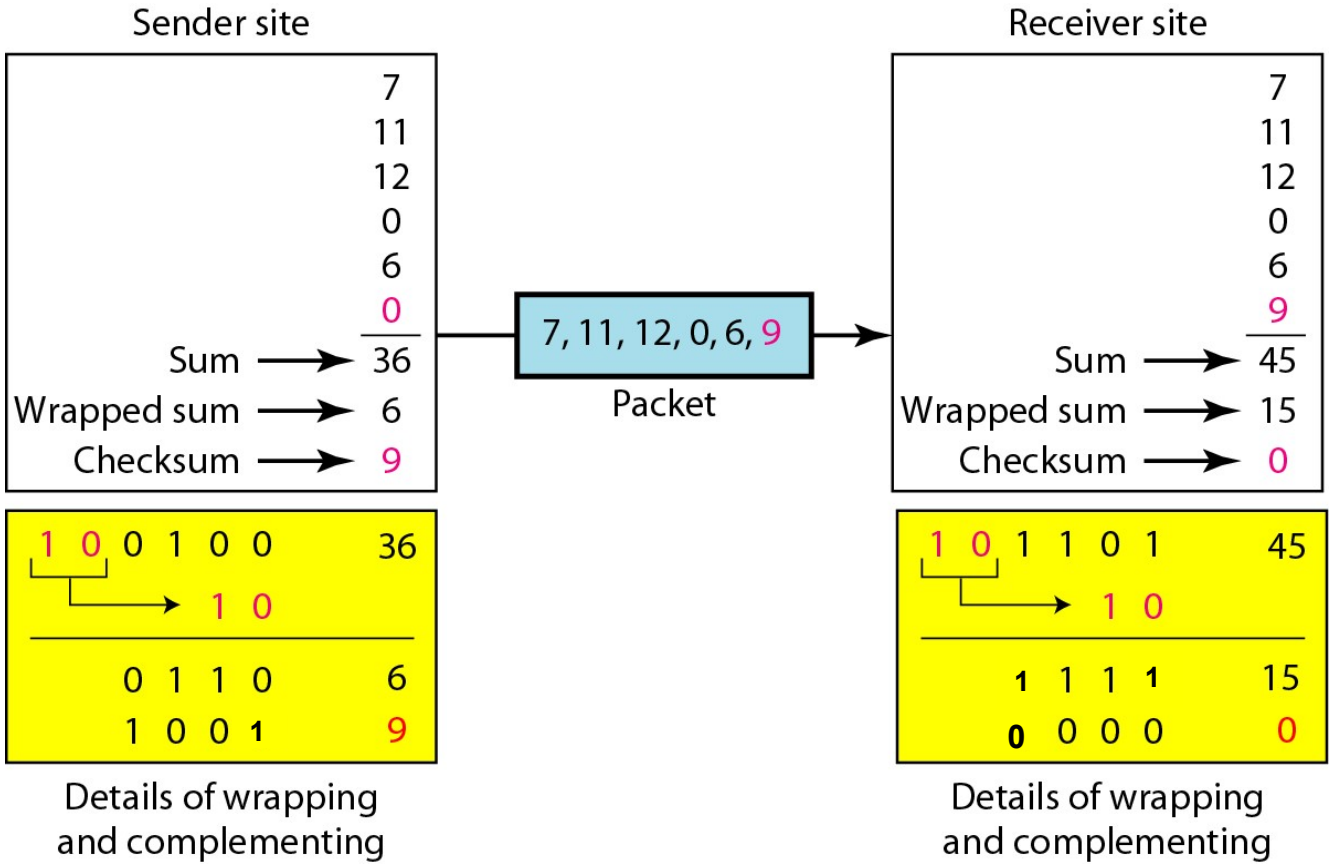

At the destination, the checker creates a new checksum from the combination of the message and sent checksum. If the new checksum is all 0s, the message is accepted; otherwise, the message is discarded

# *Checksum*

Suppose our data is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers.

For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 9), where 9 is the checksum.

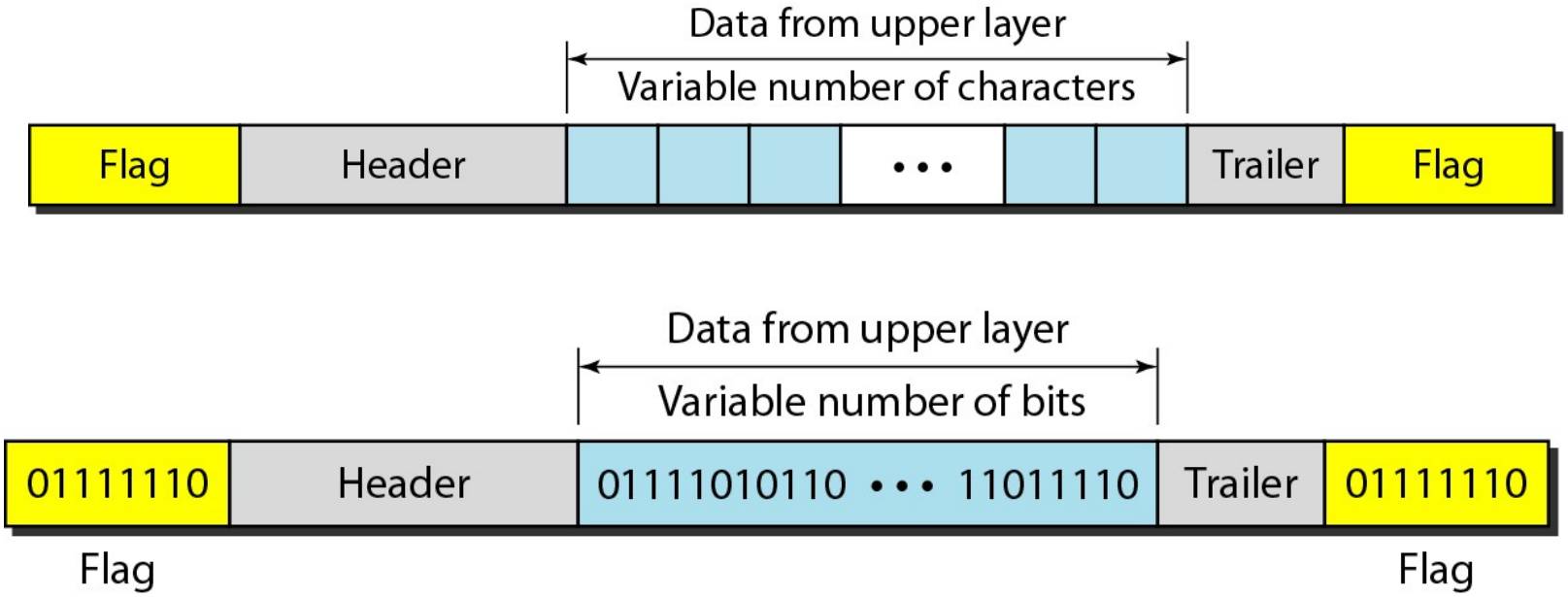
The receiver can add all the numbers received (including the checksum). If the result is 0, it assumes no error; otherwise, there is an error.



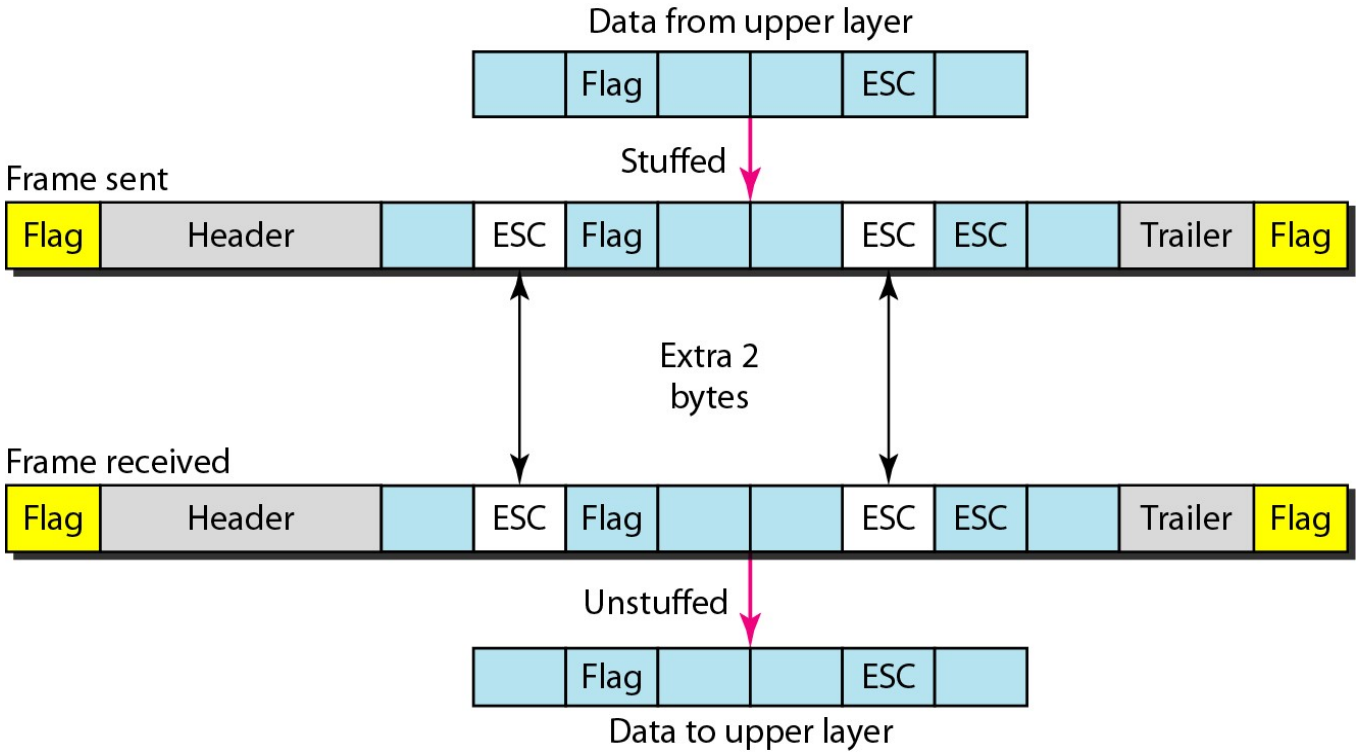
# Data Link Control

# FRAMING

*The data link layer needs to pack bits into frames, so that each frame is distinguishable from another.*



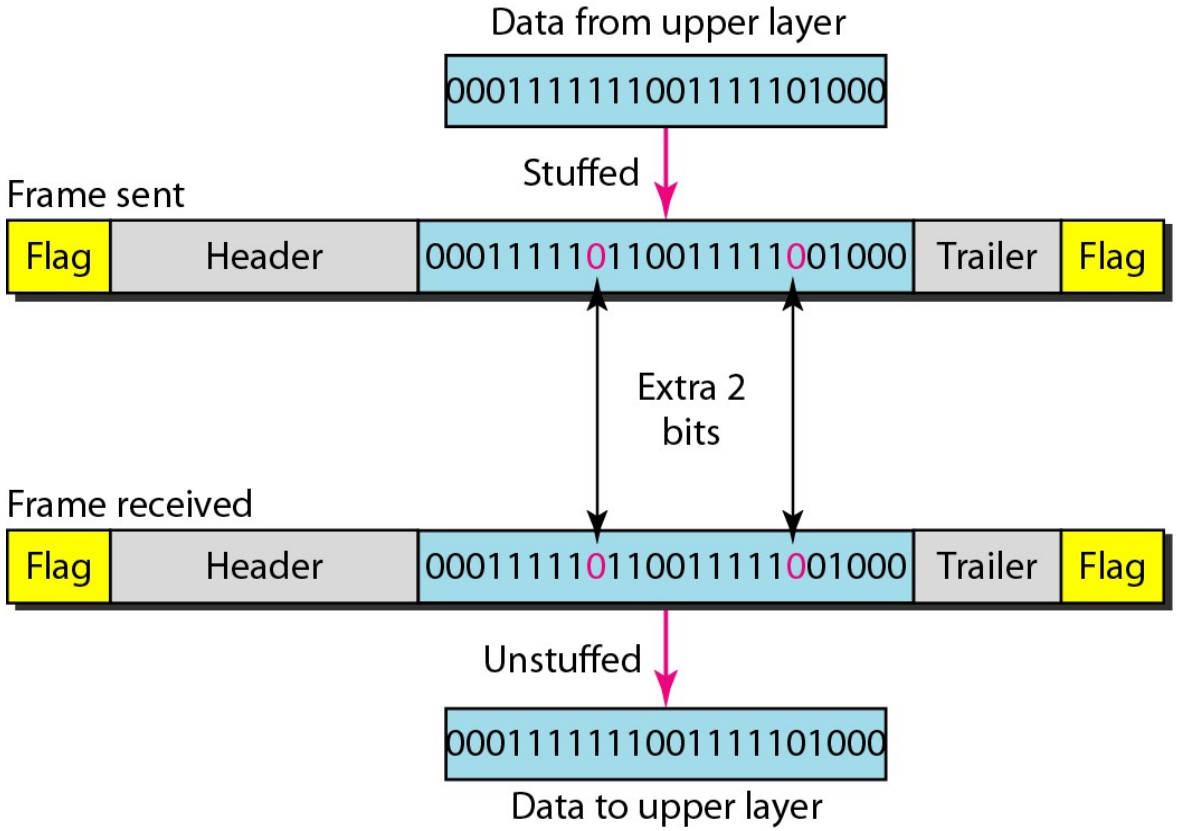
# *Byte stuffing and unstuffing*




Byte stuffing is the process of adding 1 extra byte whenever there is a flag or escape character in the text.

# *Bit stuffing and unstuffing*

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

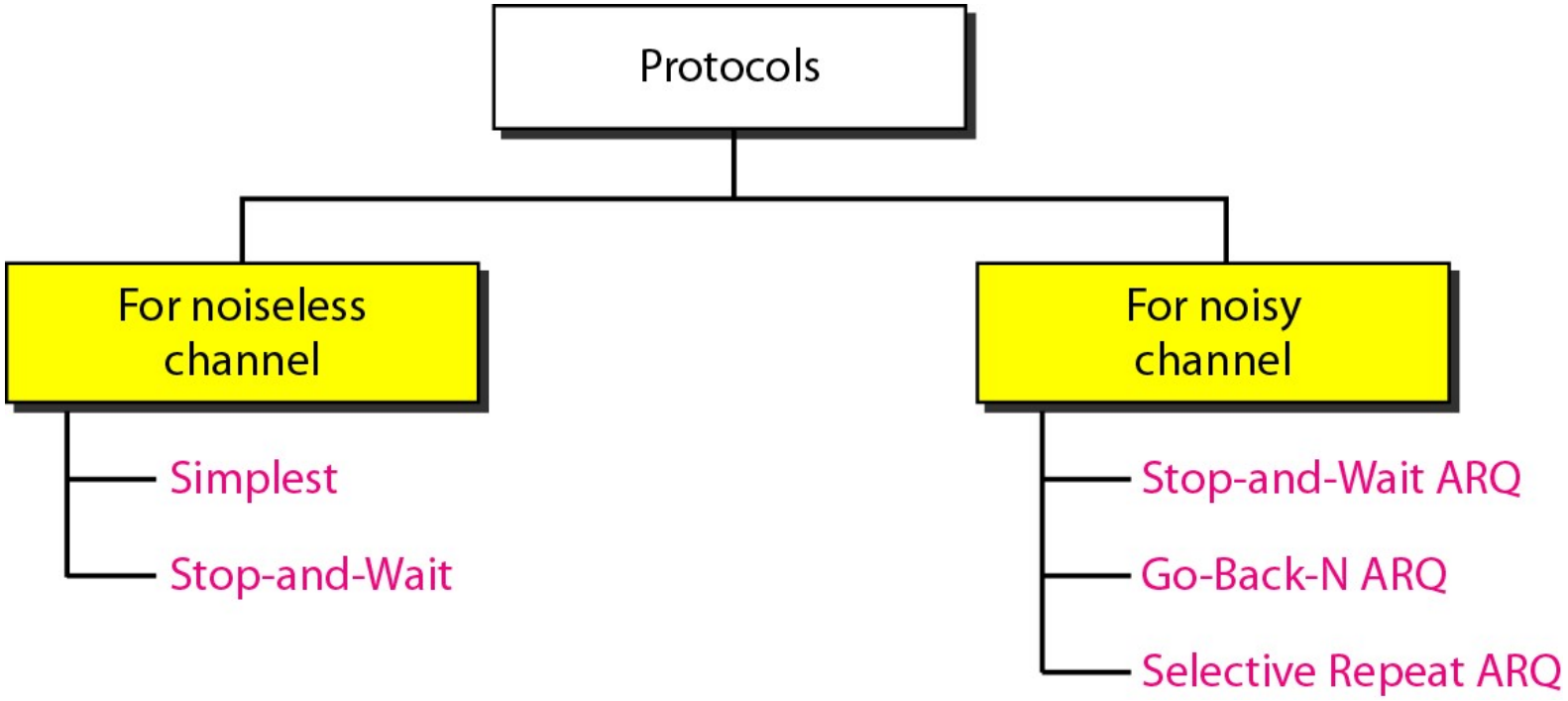


# Flow and Error Control

*The most important responsibilities of the data link layer are flow control and error control. Collectively, these functions are known as data link control.*

**Flow Control : Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.**

**Error control:** Error control in the data link layer is based on automatic repeat request, which is the retransmission of data.


Protocols
For noiseless channel
For noisy channel
Simplest
Stop-and-Wait
Stop-and-Wait ARQ
Go-Back-N ARQ
Selective Repeat ARQ

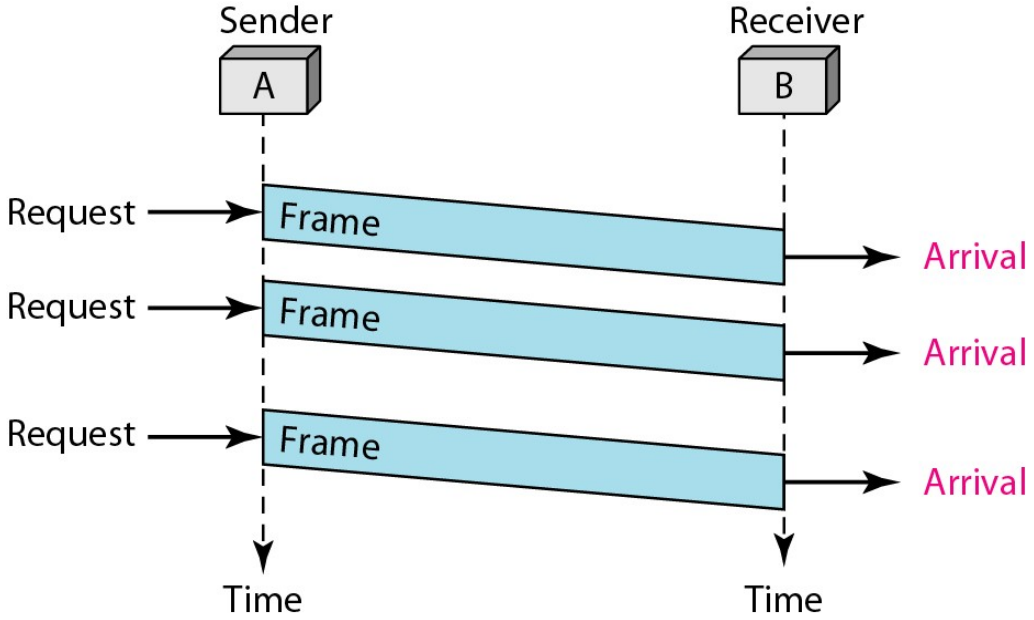
# NOISLESS CHANNELS

*Let us first assume we have an ideal channel in which no frames are lost, duplicated, or corrupted. We introduce two protocols for this type of channel.*

Simplest Protocol
Stop-and-Wait Protocol

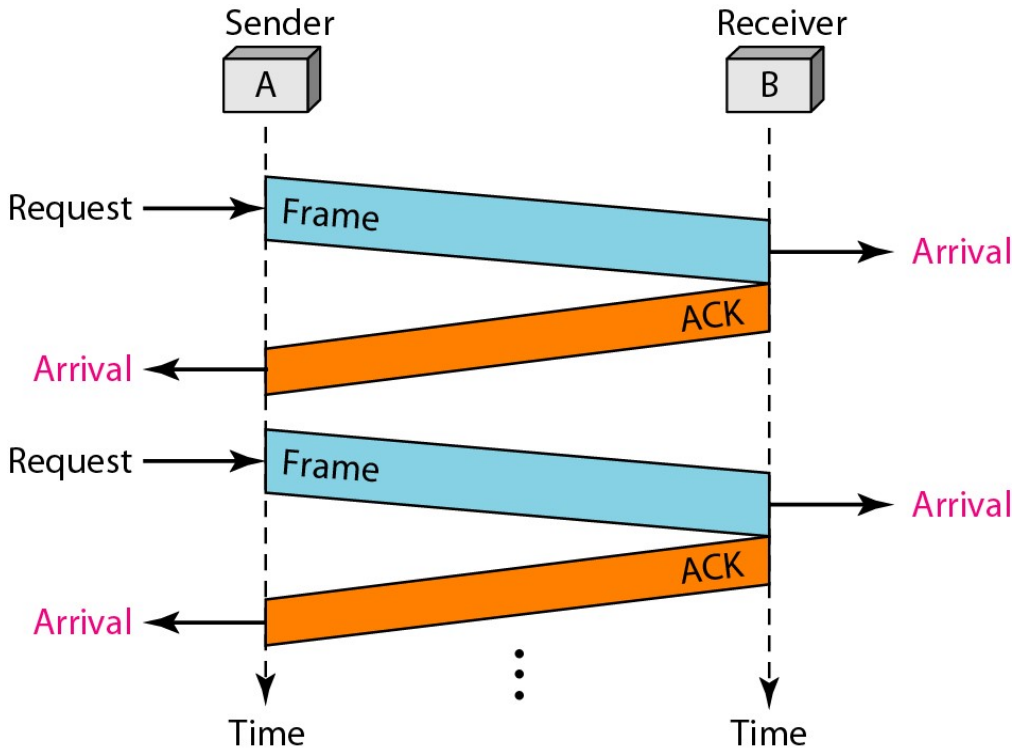
# Simplest Protocol

- Figure shows an example of communication using this protocol. It is very simple.
- The sender sends a sequence of frames without even thinking about the receiver.
- To send three frames, three events occur at the sender site and three events at the receiver site.
- Note that the data frames are shown by tilted boxes; the height of the box defines the transmission time difference between
- the first bit and the last bit in the frame.



# Stop-and-Wait Protocol

- Figure shows an example of communication using this protocol. It is still very simple.
- The sender sends one frame and waits for feedback from the receiver.
- When the ACK arrives, the sender sends the next frame.
- Note that sending two frames in the protocol involves the sender in four events and the receiver in two events.



# NOISY CHANNELS

*Although the Stop-and-Wait Protocol gives us an idea of how to add flow control to its predecessor, noiseless channels are nonexistent. We discuss three protocols in this section that use error control.*

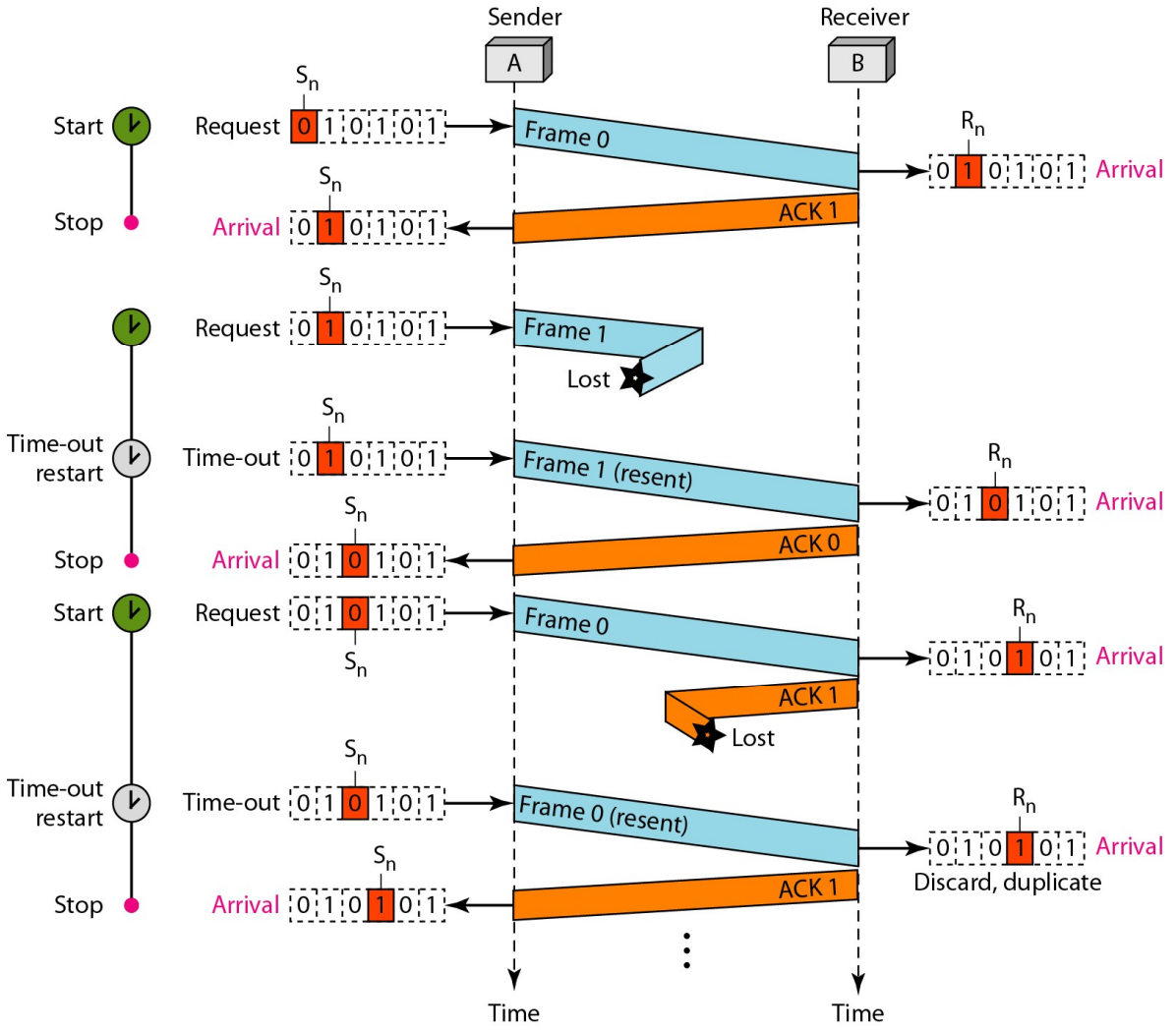
Stop-and-Wait Automatic Repeat Request
Go-Back-N Automatic Repeat Request
Selective Repeat Automatic Repeat Request

# Stop-and-Wait Automatic Repeat Request

- Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.
- In Stop-and-Wait ARQ, we use sequence numbers to number the frames. The sequence numbers are based on modulo-2 arithmetic.
- In Stop-and-Wait ARQ, the acknowledgment number always announces in modulo-2 arithmetic the sequence number of the next frame expected.

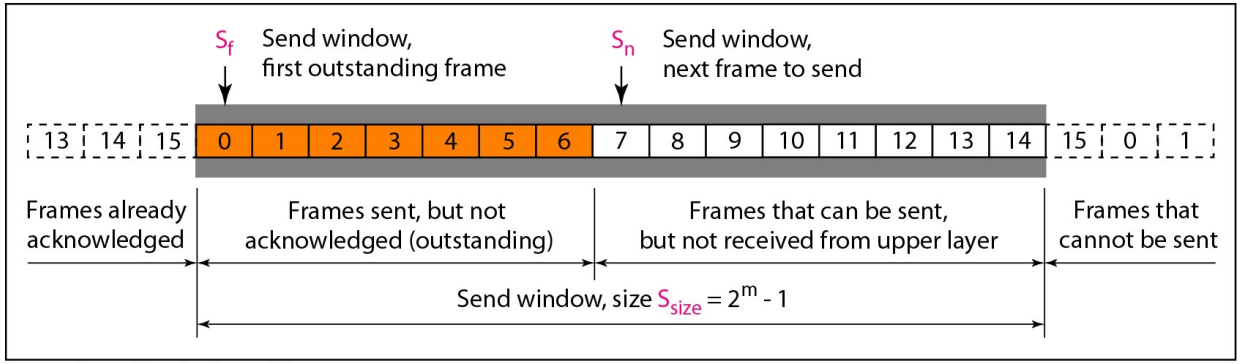
# Stop-and-Wait ARQ

- Figure shows an example of Stop-and-Wait ARQ.
- Frame 0 is sent and acknowledged.
- Frame 1 is lost and resent after the time-out. The resent frame 1 is acknowledged and the timer stops.
- Frame 0 is sent and acknowledged, but the acknowledgment is lost.
- The sender has no idea if the frame or the acknowledgment is lost, so after the time-out, it resends frame 0, which is acknowledged.

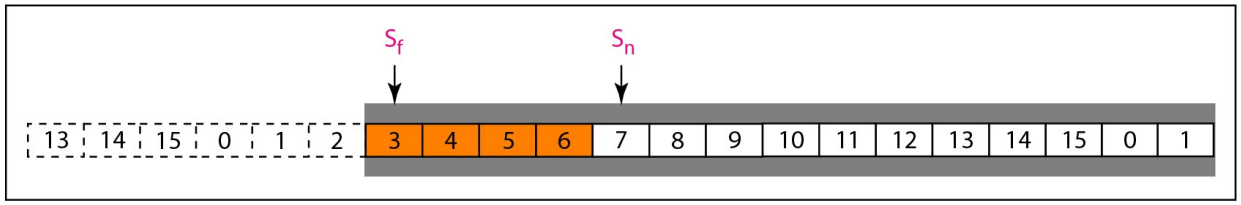


# *Go-Back-N ARQ*

- In the Go-Back-N Protocol, the sequence numbers are modulo $2^m$, where m is the size of the sequence number field in bits.
- The send window is an abstract concept defining an imaginary box of size $2^m - 1$ with three variables: Sf, Sn, and $S_{size}$.
- The receive window is an abstract concept defining an imaginary box of size 1 with one single variable $R_n$.



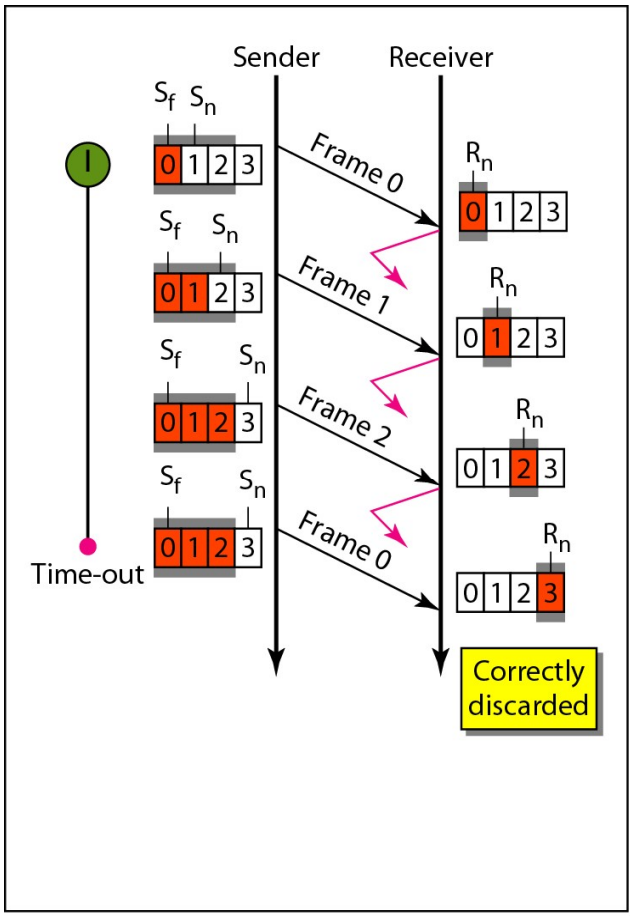

a. Send window before sliding



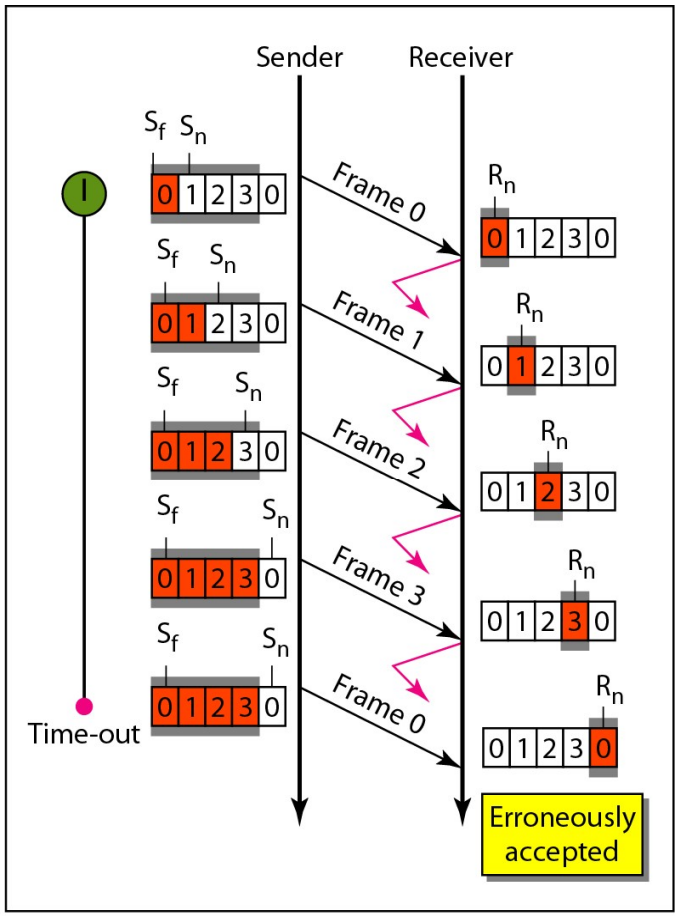

b. Send window after sliding

# *Window size for Go-Back-N ARQ*

- In Go-Back-N ARQ, the size of the send window must be less than $2^m$ the size of the receiver window is always 1.




a. Window size < $2^m$



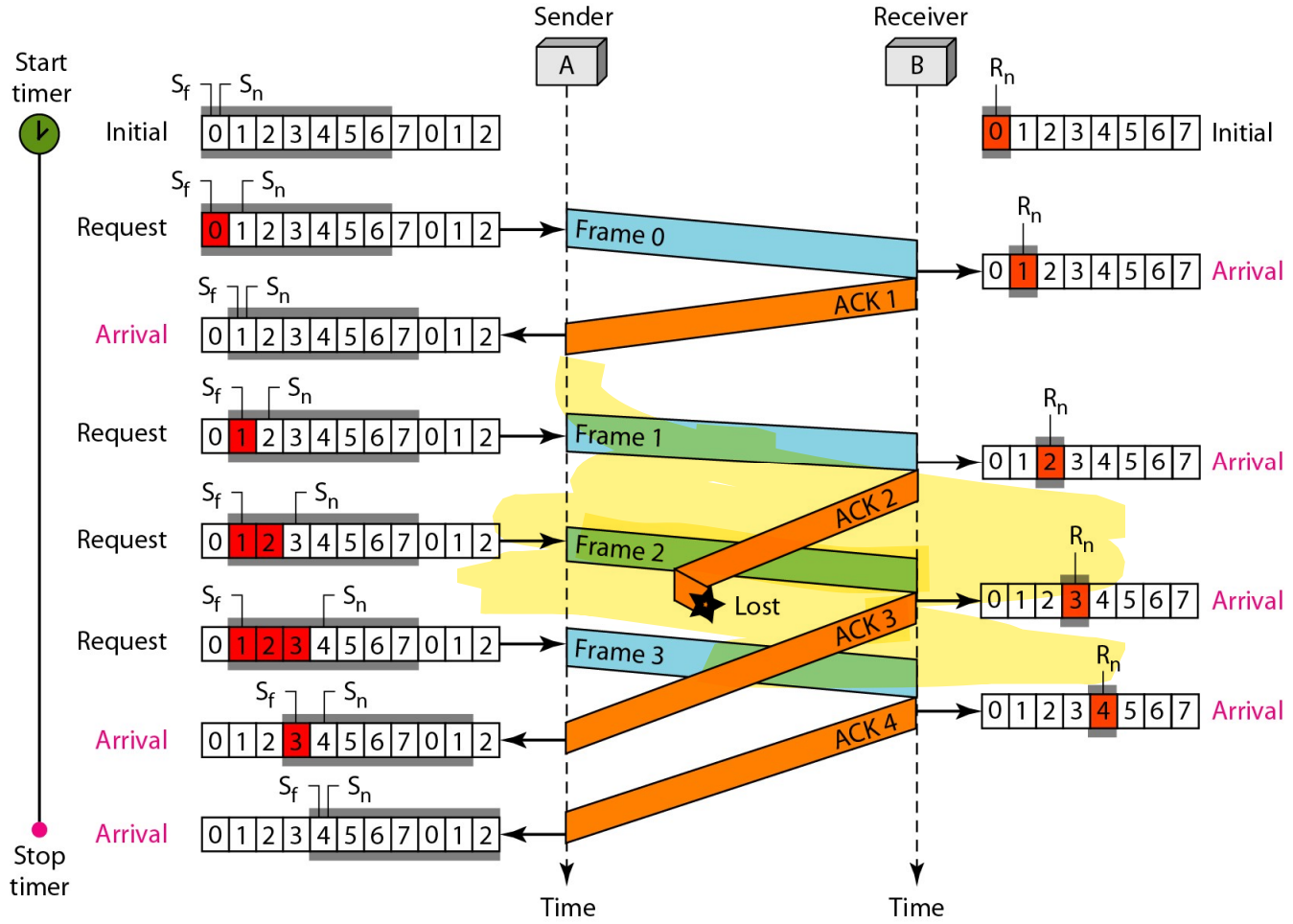

b. Window size = $2^m$

# *Flow diagram*

Figure shows an example of Go-Back-N. After initialization, there are seven sender events.

No data frames are lost, but some ACKs are delayed and one is lost. There is no time-out event here because all outstanding frames are acknowledged before the timer expires.

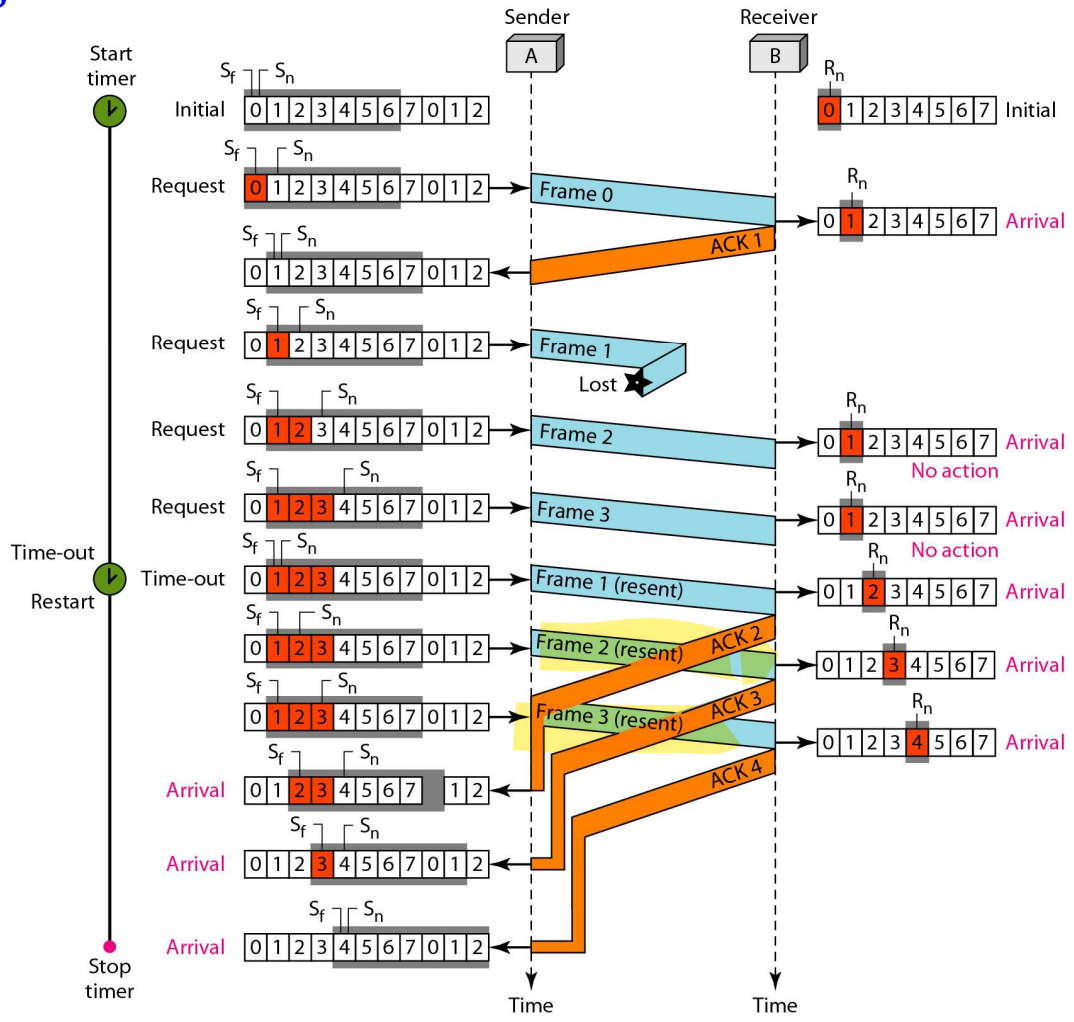
The example how cumulative acknowledgments can help if acknowledgments are delayed or lost.



# *Flow diagram*

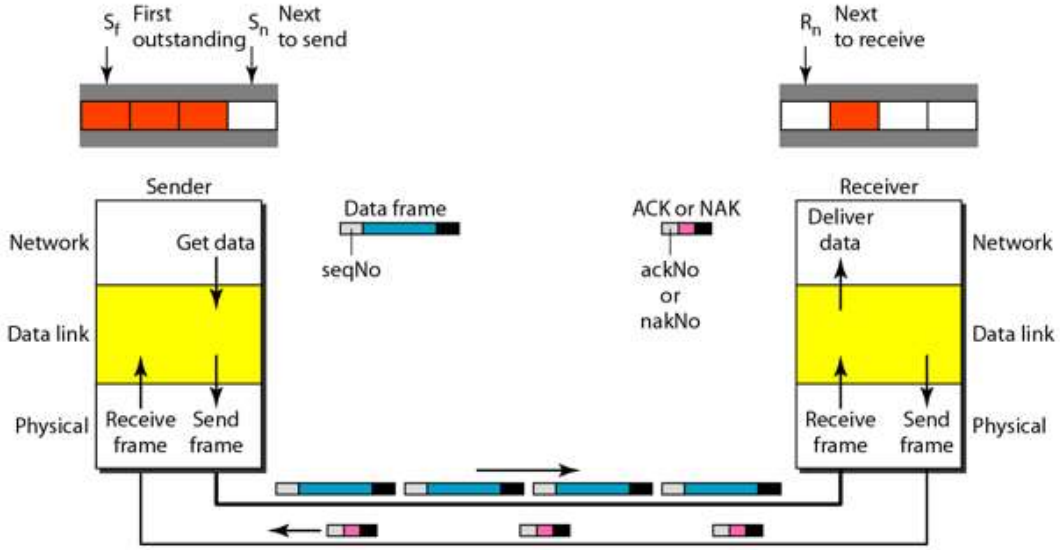
Figure shows what happens when a frame is lost. Frames 0, 1, 2, and 3 are sent. However, frame 1 is lost. The receiver receives frames 2 and 3, but they are **discarded** because they are received **out of order.**

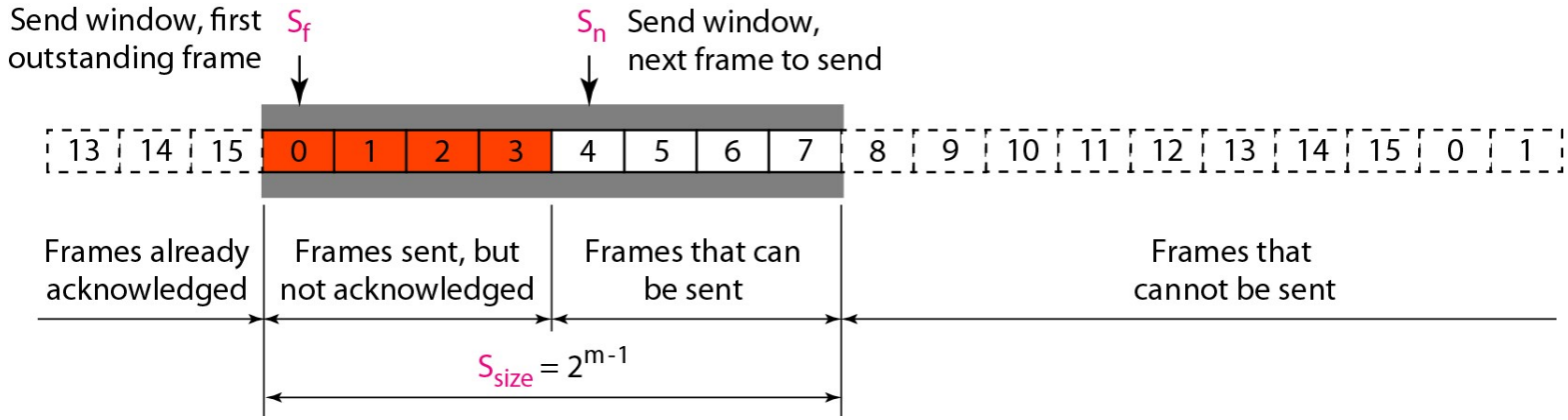
The sender receives no acknowledgment about frames 1, 2, or 3. Its timer finally expires. The sender sends all outstanding frames (1, 2, and 3) because it does not know what is wrong. Note that the resending of frames 1, 2, and 3 is the response to one single event.



# *Selective Repeat ARQ*

In Selective Repeat ARQ, the size of the sender and receiver window must be at most **one-half of $2^m$**

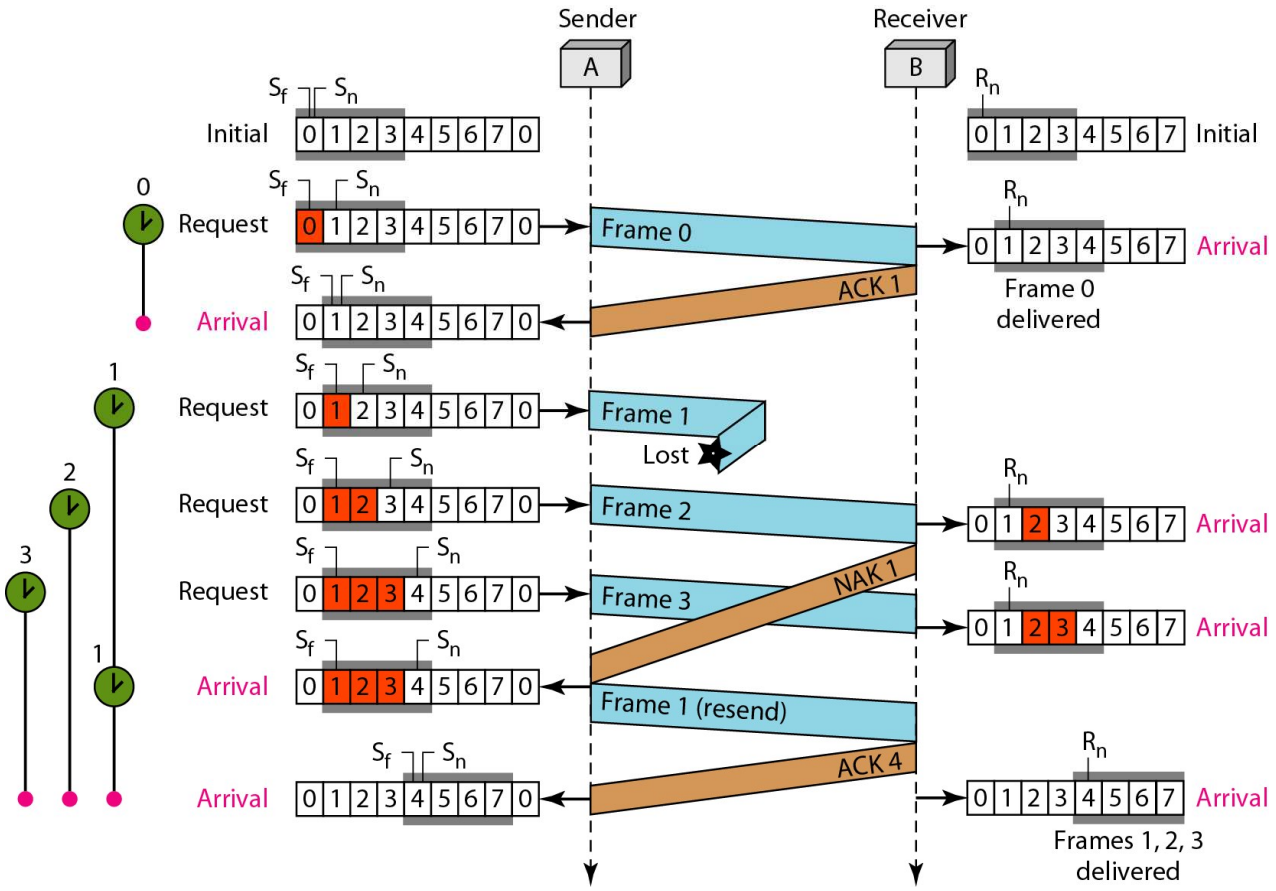


# *Flow diagram*

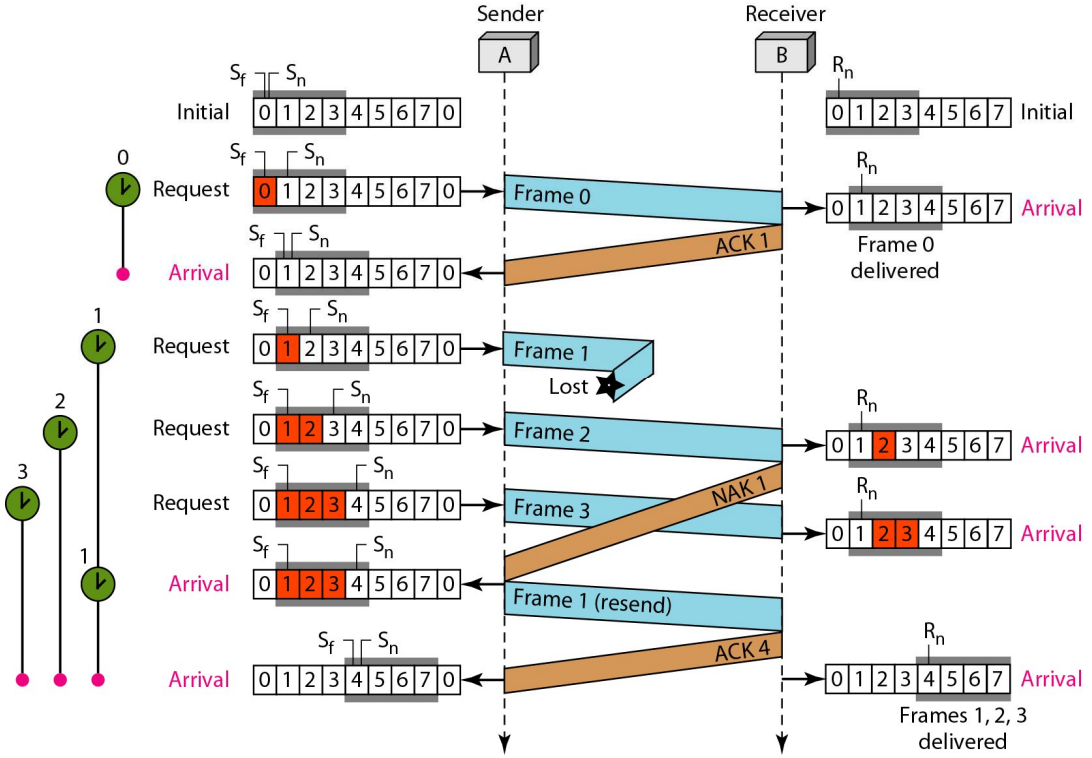
*Example in which frame 1 is lost. Here, each frame sent or resent needs a timer, which means that the timers need to be numbered (0, 1, 2, and 3). The timer for frame 0 starts at the first request, but stops when the ACK for this frame arrives.*

*The timer for frame 1 starts at the second request, restarts when a NAK arrives, and finally stops when the last ACK arrives. The other two timers start when the corresponding frames are sent and stop at the last arrival event.*



# *Flow diagram*

- At the receiver site, frame 2 and frame 3 arrive and are stored and marked, but it cannot be delivered because frame 1 is missing. When finally, a copy of frame 1 arrives, frames 1, 2, and 3 be delivered to the network layer.
- Another important point is that a NAK is sent after the second arrival, but not after the third. The reason is that the protocol does not want to crowd the network with unnecessary.




- The next point is about the ACKs. Notice that only two ACKs are sent here. The first one acknowledges only the first frame; the second one acknowledges three frames. In Selective Repeat, ACKs are sent when data are delivered to the network layer. If the data belonging to n frames are delivered in one shot, only one ACK is sent for all of them.

# HDLC

*High-level Data Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links. It implements the ARQ mechanisms.*

*Topics discussed in this section:*

Configurations and Transfer Modes
Frames
Control Field

# Normal Response Mode(NRM)

- In normal response mode(NRM), the station configuration is unbalanced.
- One primary station and multiple secondary stations are present.
- A primary station can send commands, a secondary station can only respond.
- The NRM is used for both point-to-point and multiple-point link.

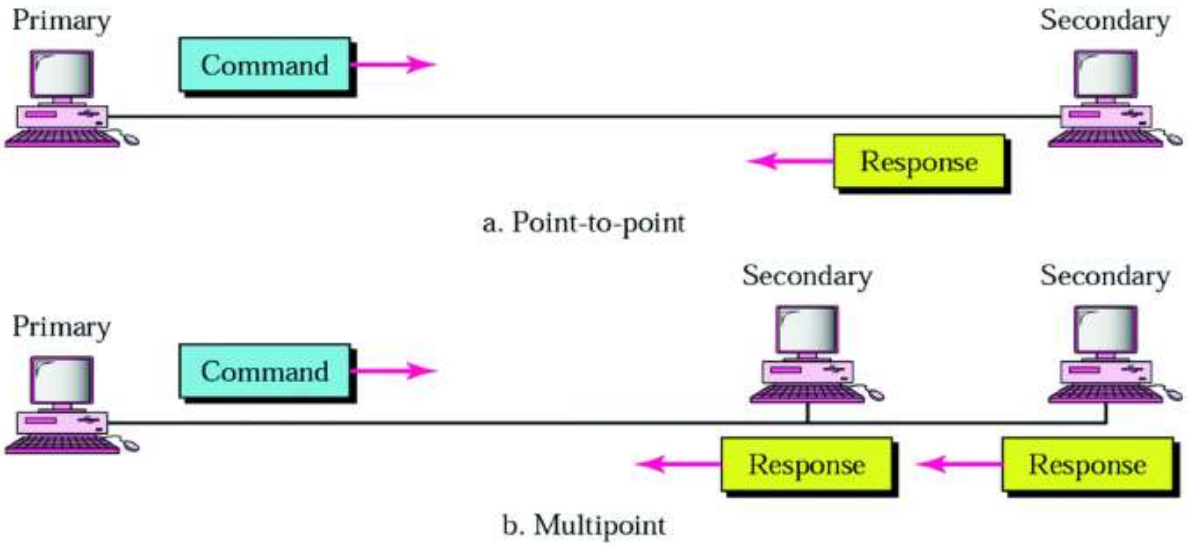



*Figure:* Normal Response Mode(NRM)

# Asynchronous Balanced Mode (ABM)

In this mode, the configuration is balanced. The link is point-to-point and each station can function as a primary and a secondary.

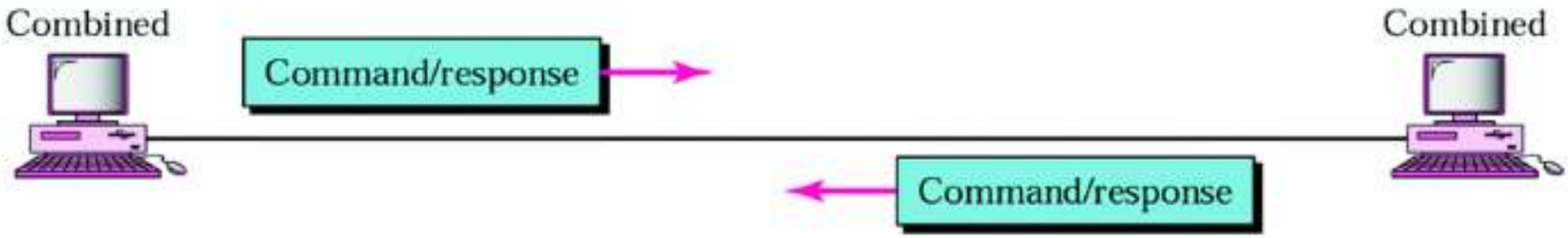


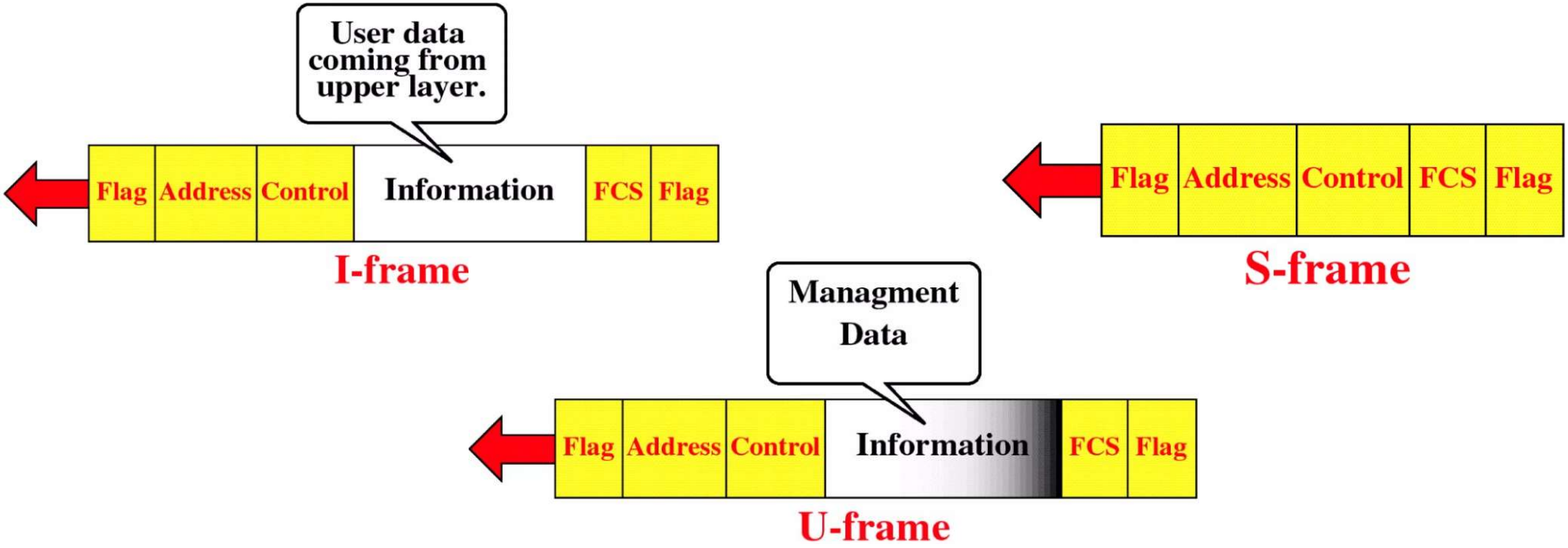

*Figure:* Asynchronous Balanced Mode(ABM)

# HDLC Frames

- HDLC defines three types of frames: information frames (I-frames), supervisory frames (S-frames) and unnumbered frames (U-frames).
- I-frames are used to transport user data and control information relating to user data.
- S-frames are used only to transport control information.
- U-frames are reserved for system management.

# HDLC Frame Format

Each frame in HDLC may contain up to six fields as shown in figure: a beginning flag field, an address field, a control field, an information field, a frame check sequence (FCS) field, and an ending flag field.

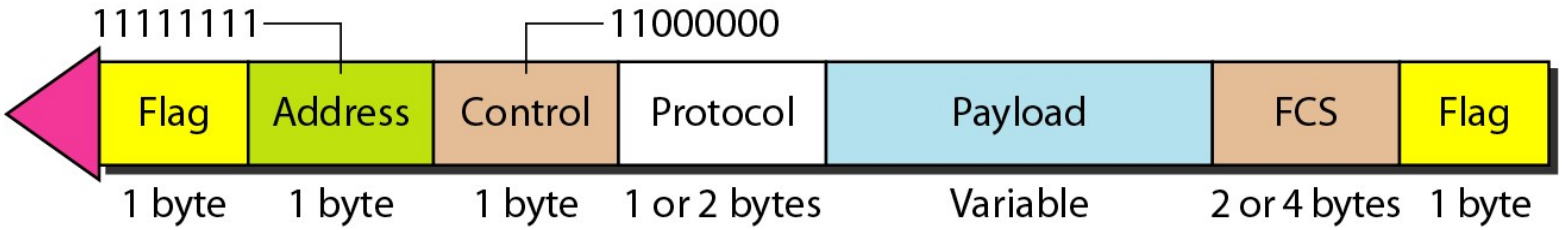


# *Fields*

- ***Flag field:*** The flag field of an HDLC frame is an 8-bit sequence with the bit pattern 01111110 that identifies both the beginning and the end of a frame and serves as a synchronization pattern of the receiver.
- ***Address field:*** The second field of an HDLC frame contains the address of the secondary station.
- ***Control field:*** The control field is a 1 or 2-byte segment of the frame used for flow and error control. This field determines the type of frames and defines its functionality
- ***Information field:*** This field contains the user's data from the network layer or management information.
- ***FCS field:*** The frame check sequence (FCS) is the HDLC error detection field. It can contain either a 2- or 4-byte.

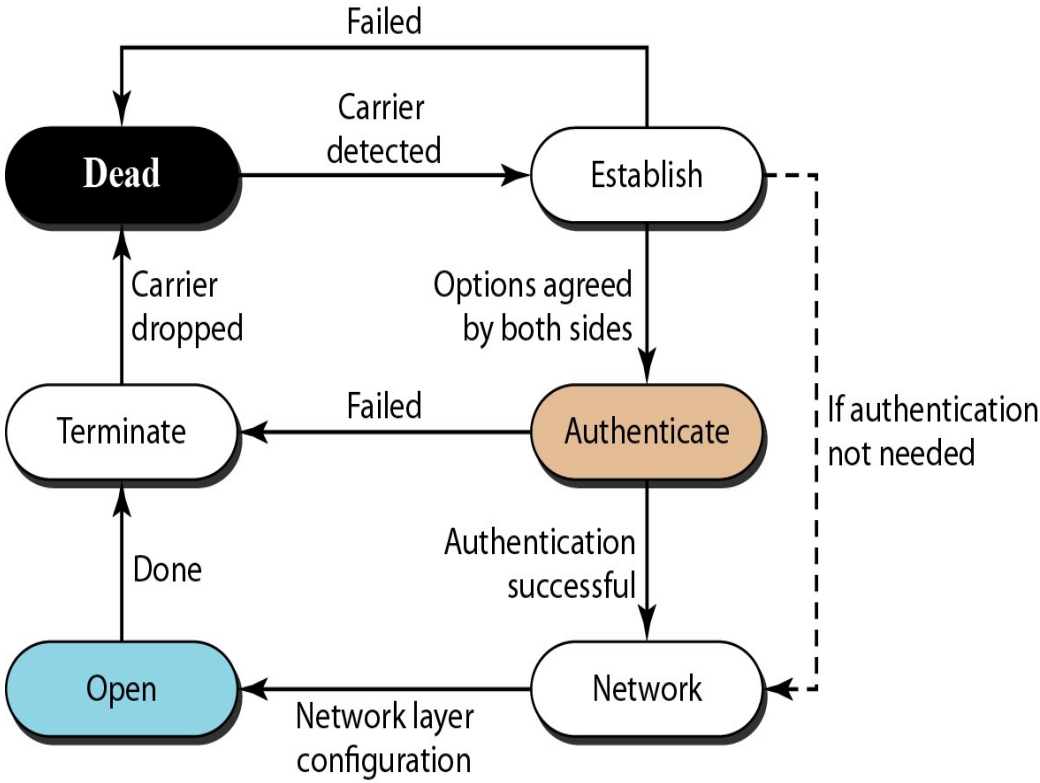
# POINT-TO-POINT PROTOCOL

*Although HDLC is a general protocol that can be used for both point-to-point and multipoint configurations, one of the most common protocols for point-to-point access is the Point-to-Point Protocol (PPP). PPP is a byte-oriented protocol.*



# POINT-TO-POINT PROTOCOL

- **Dead:** There is no active carrier (at the physical layer) and the line is quiet.
- **Establish:** When one of the two nodes starts the communication. In this state, options are negotiated between the two parties. If the two parties agree that they need authentication, then the system needs to do authentication (an extra step); otherwise, the parties can simply start communication.
- Data transfer takes place in the open state. When a connection reaches this state, the exchange of data packets can be started.




The connection remains in this state until one of the endpoints wants to terminate the connection. In this case, the system goes to the terminate state. The system remains in this state until the carrier (physical-layer signal) is dropped, which moves the system to the dead state again.